

A Task Specification

OR-Agent is designed to operate over a broad class of research tasks that can be formulated as program synthesis and automated evaluation problems. The framework introduces a unified task specification interface that bridges human intent, large language models, and executable evaluation environments. To enable automated research, users must decompose their target problem into a structured specification that defines the problem semantics, evaluation methodology, and interface constraints governing the evolutionary search process. This specification comprises several core components: a textual problem description, a function description specifying the target function to be evolved, an evaluation script, an evaluation description, and a callbacks description. The evaluation description may coincide with the evaluation script for simple cases or abstract essential semantics for complex or proprietary settings. The callbacks description is optional but recommended for problems with complex environments like traffic simulator. Beyond these core components, the task specification may include seed solutions (initial implementations to bootstrap the evolutionary process), dataset generation scripts (for problems requiring test instances), and external knowledge resources (domain-specific reference materials). Together, these elements define a standardized research interface that enables controlled and extensible automated experimentation.

Open Research Canvas (OR-Canvas) To facilitate flexible and user-centric task formulation, OR-Agent introduces the *Open Research Canvas (OR-Canvas)* (See Figure 1 (a)), an interactive interface that allows users to collaboratively specify research problems with the assistance of large language models. OR-Canvas supports the incremental construction of problem descriptions, function signatures, evaluation criteria, and environment interaction protocols. The OR-Canvas functions as a persistent specification artifact that grounds subsequent automated exploration, helping maintain alignment between human intent and agent-driven research while reducing reliance on ad hoc prompt engineering.



Figure 1: User Interface of OR-Canvas and OR-Agent.

B Metrics, Feature, and Score Definitions for the Cooperative Driving Problem

This section describes the evaluation protocol used for the Cooperative Driving Problem. The evaluation script returns three components: raw metrics extracted from the simulator, feature representations for diversity preservation, and a scalar fitness score for optimization. It illustrates how metrics, features, and scores are formally defined to enable automated problem evaluation.

1. **Raw metrics** extracted from the simulation environment,
2. **Feature vectors** derived from metrics for diversity preservation,
3. A scalar **fitness score** used for optimization.

Together, these components define the optimization landscape explored by OR-Agent.

B.1 Raw Evaluation Metrics

Each candidate driving policy is evaluated in the SUMO simulator under multiple traffic scenarios. The simulator returns aggregated metrics per test case. An example output is shown below:

Listing 1: Example evaluation metrics returned by the simulator.

```
{
  'critical_ttc_count': 28,
  'collisions': 0,
  'emergencyStops': 0,
  'emergencyBraking': 4,
  'teleports': 0,
  'avg_fuel_consumption': 8.32,
  'avg_speed': 12.51,
  'speed_variance': 16.22
}
```

The metrics are grouped into three categories:

- **Safety indicators:** collisions, emergency stops, emergency braking events, teleports (simulation failures), and critical time-to-collision (TTC) counts.
- **Efficiency indicators:** average speed and fuel consumption.
- **Smoothness indicators:** speed variance.

Metrics are aggregated across multiple test scenarios before further processing.

B.2 Feature Vector Construction

Feature vectors are used for diversity preservation in the solution database (e.g., in MAP-Elites style clustering). Unlike scalar fitness, features characterize behavioral signatures rather than overall quality.

Behavioral Signature (Recommended) The recommended feature representation discretizes aggregated metrics into interpretable categorical levels representing safety, efficiency, and smoothness. Each dimension is mapped to a discrete scale:

$$\text{Signature} = (\text{safety level}, \text{speed efficiency level}, \text{smoothness level}),$$

where each level takes values in $\{0, 1, 2, 3\}$ (0 for “poor”; 1 for “weak”; 2 for “good”; 3 for “excellent”).

Safety level is primarily determined by collisions and teleports, followed by emergency events and critical TTC counts. **Speed efficiency level** measures deviation from a predefined urban target speed. **Smoothness level** is determined by speed variance, with lower variance corresponding to higher smoothness.

This discretization yields interpretable behavioral clusters such as: *Safe-Conservative-Smooth* or *Risky-Fast-Aggressive*. The granularity of discretization determines the potential diversity resolution of the archive.

Alternative Representations Other feature construction strategies are also supported:

- **Continuous normalized features:** for clustering methods such as k-means,
- **Risk-profile signatures:** emphasizing safety-performance trade-offs.

OR-Agent does not enforce a fixed representation; feature design is treated as part of task specification.

B.3 Scalar Score Calculation

While feature vectors maintain diversity, optimization is guided by a scalar fitness score. The scoring function reflects three prioritized objectives: **Safety**, **Speed efficiency**, and **Driving smoothness**. For each test case, three sub-scores are computed.

Safety Score Safety begins with a baseline of 100 and receives additive penalties:

$$S_{\text{safety}} = 100 - 50 \cdot \text{collisions} - 30 \cdot \text{teleports} \\ - 5 \cdot \text{emergencyStops} - 2 \cdot \text{emergencyBraking} - 0.5 \cdot \text{criticalTTC}.$$

Collisions and teleports incur the most severe penalties, reflecting their critical importance.

Speed Efficiency Score Speed efficiency is evaluated based on the deviation of the average driving speed v from a predefined urban target speed $v^* = 13.89$ m/s (50 km/h). Scores decay piecewise as deviation increases:

$$S_{\text{speed}} = f(|v - v^*|),$$

where $f(\cdot)$ is a monotonic decreasing function. Maximum score is achieved when average speed is close to the target range.

For the current implementation of OR-Agent, we adopt the following piecewise linear function:

$$S_{\text{speed}} = \begin{cases} 100, & |v - v^*| \leq 1.5, \\ 100 - 5(|v - v^*| - 1.5), & 1.5 < |v - v^*| \leq 3.5, \\ \max(0, 90 - 8.662(|v - v^*| - 3.5)), & |v - v^*| > 3.5. \end{cases}$$

Smoothness Score Driving smoothness is evaluated using the variance of vehicle speed, denoted by σ_v^2 . Lower speed variance indicates smoother driving behavior. The smoothness score is computed as:

$$S_{\text{smoothness}} = g(\text{speed variance}),$$

where $g(\cdot)$ is a piecewise linear decreasing function. Lower variance yields higher smoothness scores.

For the current implementation of OR-Agent, we adopt the following piecewise linear function:

$$S_{\text{smoothness}} = \begin{cases} 100, & \sigma_v^2 \leq 5.0, \\ 100 - 4(\sigma_v^2 - 5.0), & 5.0 < \sigma_v^2 \leq 10.0, \\ \max(0, 80 - 8(\sigma_v^2 - 10.0)), & \sigma_v^2 > 10.0. \end{cases}$$

Figure 2 illustrates the piecewise implementations of the speed efficiency and driving smoothness score functions adopted in the current version of OR-Agent.

Final Combined Score The final score for each test case is a weighted combination:

$$\text{Score} = 0.5 \cdot S_{\text{safety}} + 0.3 \cdot S_{\text{speed}} + 0.2 \cdot S_{\text{smoothness}}.$$

Safety receives the highest weight, reflecting its dominance in real-world driving systems. Speed efficiency and smoothness serve as secondary objectives once safety constraints are satisfied.

B.4 Design Principles

The evaluation design follows three guiding principles:

1. **Safety dominance.** Severe penalties ensure unsafe policies are strongly disfavored.
2. **Targeted efficiency.** Policies are rewarded for maintaining realistic urban speeds.
3. **Stability preference.** Lower speed variance promotes smooth traffic flow.

By separating behavioral features from scalar fitness, OR-Agent simultaneously encourages diversity exploration and objective-driven optimization.

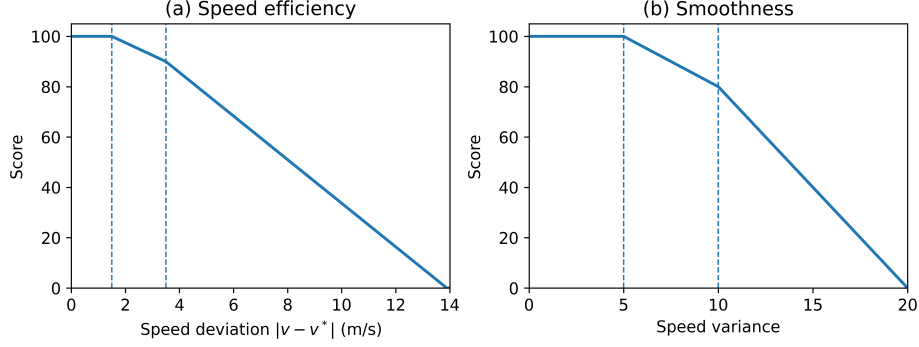


Figure 2: Piecewise scoring functions adopted in the current version of OR-Agent: (a) speed efficiency score as a function of speed deviation from the target speed and (b) driving smoothness score as a function of speed variance.

Remark The definitions of metrics, features, and scoring functions fundamentally shape the optimization landscape. OR-Agent deliberately exposes these definitions as configurable components rather than hard-coded constraints, enabling domain experts to tailor objective trade-offs according to practical requirements.

C Coordinated Idea Generation vs. Independent Idea Generation Example

This section compares two idea generation strategies used during research tree expansion: *independent generation* and *coordinated generation*. Empirical observations suggest that the choice of generation strategy significantly affects diversity, coverage, and systematic exploration.

C.1 Independent Idea Generation

In the independent generation setting, child ideas are produced by invoking the LLM multiple times separately, each time conditioned only on the parent solution. Even when increasing temperature (e.g., up to 1.3), the diversity of generated ideas remains surprisingly limited. As an illustrative example, consider the Online Bin Packing Problem (BPP-Online). Twenty ideas were independently generated from the same parent solution. A representative sample is shown in Listing 2.

Listing 2: Twenty independently generated child ideas for BPP-Online from the same parent solution.

1. Combine value-to-weight ratio with a sparsification mechanism that zeros out items whose total weight exceeds a dynamic threshold based on average knapsack capacity usage. Specifically, compute the initial heuristic as prize divided by the L1 norm of weight (as in the elite solution), then set to zero any item whose max weight across constraints exceeds $1/\sqrt{m}$ (reflecting tighter per-dimension feasibility), or alternatively, zero out the bottom 30% of items by this ratio to focus search on most promising candidates.
2. Combine value-to-weight ratio with a sparsification mechanism that zeros out items whose total weight exceeds the average capacity usage per item, to focus selection on both efficient and feasible items. Specifically, compute the initial heuristic as prize divided by the L1 norm of weight (as in the elite solution), then set to zero any item whose total weight (sum across constraints) is greater than the median total weight of all items, thereby promoting inclusion of lighter, high-value items while pruning potentially infeasible heavy ones early.
3. Combine value-to-weight ratio with a sparsification mechanism that zeros out items whose total weight exceeds a dynamic threshold based on average knapsack capacity usage. Specifically, compute the initial heuristic as prize divided by the sum of weights (as in the elite solution), then set to zero any item whose maximum weight across all constraints exceeds $1/m$ (the average per-constraint capacity), making it less likely to be selected due to high risk of violating constraints.
4. Combine value-to-weight ratio with a sparsification mechanism that zeros out items whose total weight exceeds the average capacity usage per item. Specifically,

```

compute the standard value-to-total-weight ratio (prize / sum(weight, axis=1)),
then identify items with total weight greater than the median total weight across
all items; set their heuristic value to zero to discourage selection of overly
heavy items unless they are exceptionally valuable. This balances desirability
with feasibility under multiple constraints.
5. Combine value-to-weight ratio with a sparsification mechanism that zeros out items
whose total weight exceeds the average capacity pressure across constraints.
Specifically, compute the initial heuristic as prize divided by the L1 norm of
weight (as in the elite solution), then set to zero any item where the maximum
weight across constraints exceeds a threshold (e.g., 1/m or the median of max
weights). This focuses ant attention on items that are both valuable per unit
resource and feasible under tightest constraints.
...(Check public repository for the complete list)...

```

Despite nominal stochastic variation, most ideas follow nearly identical structural patterns:

- Variants of prize-to-weight ratio heuristics,
- Minor modifications of sparsification thresholds,
- Slight adjustments of L1 vs L2 norms,
- Similar median-based pruning mechanisms.

The dominant theme—“combine value-to-weight ratio with sparsification”—appears repeatedly with minimal conceptual deviation. This phenomenon indicates that independent sampling, even at elevated temperature, tends to produce *locally perturbed variations* rather than structurally distinct hypotheses.

C.2 Coordinated Idea Generation

In contrast, OR-Agent employs *coordinated generation*, where all child ideas of a node are produced in a single LLM invocation. The model is explicitly instructed to generate multiple *distinct* directions simultaneously. Using the same BPP-Online parent solution, twenty coordinated ideas are shown in Listing 3.

Listing 3: Twenty coordinated child ideas for BPP-Online generated in a single LLM call.

```

1. Normalize the prize-to-weight ratio per constraint dimension and aggregate via
   geometric mean to better reflect balanced resource usage across all constraints.
2. Introduce a sparsification step that sets heuristic values to zero for items whose
   total weight exceeds a threshold (e.g., average capacity usage), focusing
   selection on feasible candidates.
3. Incorporate a penalty term based on the maximum weight of an item across all
   constraints to discourage selection of items that dominate any single resource.
4. Compute heuristic as prize divided by the L2 norm of the weight vector,
   emphasizing compactness in multi-dimensional weight space.
5. Use min-max normalization of weights per dimension before computing
   prize-to-weight ratios to ensure equitable contribution from each constraint.
...(Check public repository for the complete list)...

```

Compared to independent generation, coordinated generation exhibits:

- Substantially reduced redundancy,
- Broader heuristic families (geometric mean aggregation, surrogate LP relaxation, harmonic mean weighting, ensemble heuristics, diversity factors),
- Structural diversity rather than threshold tuning.

This behavior resembles the scientific practice of enumerating multiple conceptual improvement directions before experimental validation. Rather than sampling near-identical local perturbations, the model produces a *systematic hypothesis set* covering multiple design axes.

C.3 Advantages and Trade-offs

Advantages of Coordinated Generation

1. **Improved diversity.** Ideas exhibit greater conceptual separation.
2. **Better coverage.** Exploration spans multiple heuristic paradigms.
3. **Systematic reasoning.** The model implicitly allocates different “slots” to distinct improvement themes.

Limitations Coordinated generation is not without drawbacks:

- Individual ideas tend to be shorter when many are generated simultaneously.
- Explicitly constraining word count per idea is possible but inelegant, as optimal length is task-dependent.
- Generating many ideas in one call is constrained by the context window.

Thus, coordinated generation introduces a trade-off between diversity and per-idea elaboration.

C.4 Implications for Research Tree Expansion

Within OR-Agent, each research tree node expands via coordinated generation. All child hypotheses are produced together, evaluated separately, and selectively retained based on performance. This design reduces redundant branching and improves structural exploration efficiency. While independent generation can be seen as Monte Carlo sampling in local neighborhoods, coordinated generation resembles structured hypothesis decomposition. Empirical results across multiple problems indicate that coordinated generation improves exploration breadth without significantly increasing computational overhead, thereby contributing to more stable research progression.

D Experiment History Compression Example

This section provides an illustrative example of experiment history compression within OR-Agent. As described in the main text, experiment reflections are progressively summarized to prevent unbounded context growth while preserving key insights for subsequent reasoning. Listing 4 shows a progressive summary generated by the `ExperimentAgent` after four experiments on cooperative driving problem. Instead of retaining all raw experiment logs verbatim, OR-Agent synthesizes structured summaries that capture performance trends, diagnostic observations, and actionable improvement directions.

Listing 4: Example of progressive experiment history compression generated by the `ExperimentAgent` after four experiments.

```
>[ExperimentAgent] Progressive summary generated:
Progressive Summary for experiments #1 ~ #4:
- Initial score: -35.132
- Current score: None
- Best score so far: -23.697

Key insights from recent experiments:
  Experiment #1: Score: -35.132 | Metrics: avg_speed: 0.430, collisions: 0.0,
    emergencyStops: 0.0, teleports: 4.5, critical_ttc_count: 155.0,
    emergencyBraking: 0.0, speed_variance: 3.595 | Insight: <insight>The latest
    experiment results show poor performance with an overall score of -35.13,
    which is worse than the parent solution’s score of -14.07. Key issues
    identified:
1. **Extremely low average speeds** (0.43 m/s vs target 13.89 m/s): Vehicles are
   moving too slowly, likely due to over-conservative acceleration and excessive
   cooperation adjustments.
2. **High critical TTC count** (155 vs 79.5): Many near-collision events indicate
   vehicles are following too closely, suggesting the IDM parameters are too
   aggressive for this traffic scenario.
3. **Teleports** (4.5): Vehicles are getting stuck and being removed by SUMO,
   indicating potential deadlocks or excessive waiting.
```

4. ****Complex negotiation system**** may be causing vehicles to slow down excessively while waiting for cooperation or lane changes.

Comparing to the parent solution #1 which had better safety (lower critical TTC) and lower teleports, we observe that our current hierarchical cooperative negotiation system might be overcomplicated. The parent solution used predictive platoon formation with anticipatory gap creation, which is simpler and more focused on maintaining flow.

However, this is only the first experiment with this solution, so we should try to improve it through parameter tuning and minor adjustments before considering termination.

Specific improvements we can make:

- Increase following distances (time headway and minimum gap) to reduce critical TTC events.
- Simplify the urgency calculation to reduce unnecessary lane changes.
- Ensure vehicles maintain minimum speeds to avoid teleports.
- Reduce the impact of cooperative adjustments to prevent excessive slowing.
- Clean up state for vehicles that have left the simulation to prevent memory issues.
- Add hysteresis to lane change decisions to reduce frequent lane changes.

We'll implement these changes through targeted code modifications.</insight>

Experiment #2: ...(omitted for brevity)...

E Learning From Failures Example

During experimentation, OR-Agent frequently encounters suboptimal or even infeasible solutions. Importantly, such failures are not discarded; instead, they serve as valuable sources of structured knowledge. The Experiment Agent is designed to extract diagnostic signals even when evaluation partially fails (e.g., timeouts or runtime errors).

The following example demonstrates this behavior in the TSP_CONSTRUCTIVE problem. In this case, the largest test instance (100 cities) timed out, yet the Experiment Agent was still able to analyze earlier results and synthesize meaningful insights regarding algorithmic behavior and scalability.

Listing 5: Example of learning from a partially failed experiment (timeout on large-scale instance).

```
>[Evaluator] lead1_round5_count0_id9 - Error: no end marker found in the result:
[*] Evaluating val20_dataset.npy
Average for 20 cities: 4.164
[*] Evaluating val50_dataset.npy
Average for 50 cities: 6.786
[*] Evaluating val100_dataset.npy
Timed out after 300 seconds

>[ExperimentAgent] Thinking: The experiment series focused on developing a hybrid
regret-lookahead heuristic for the Traveling Salesman Problem that balances
computational efficiency with global awareness. The initial implementation
combined regret-based node selection with conditional multi-step lookahead,
achieving baseline scores of 4.45 (20 cities), 7.12 (50 cities), and 9.78 (100
cities). Key improvements in Experiment #1 included:
(1) revising the regret calculation to measure opportunity cost as the difference
between second-best and best connection costs from each candidate node,
(2) adjusting the lookahead activation threshold to use median regret plus 20% of the
regret range rather than a fixed proportion,
(3) expanding end-game optimization from <=4 to <=5 remaining nodes, and
(4) improving the fallback selection strategy to simultaneously optimize for high
regret and low immediate distance.
```

These changes yielded significant performance improvements for smaller instances (20 cities: 4.16, 50 cities: 6.79), though the 100-city evaluation timed out, suggesting computational complexity remains problematic at larger scales. The most impactful parameter adjustments were the refined regret calculation methodology and the expanded end-game lookahead window. Persistent challenges include the fundamental limitation of greedy constructive approaches lacking global tour awareness and computational overhead that scales poorly with problem size. Successful strategies included using regret as a trigger mechanism for deeper analysis and incorporating progress-aware dynamic thresholds. Unresolved issues center around scalability and the inherent myopia of sequential decision-making without backtracking.

For future research, we recommend:

- (1) implementing more efficient lookahead approximations such as sampling-based future cost estimation,
- (2) integrating geometric insights like nearest neighbor clustering to reduce effective problem dimensionality,
- (3) developing adaptive regret thresholds that consider both tour progress and instance-specific characteristics, and
- (4) exploring lightweight machine learning models to predict high-value lookahead opportunities without exhaustive computation.

This behavior reflects a key design philosophy of OR-Agent: experimental failures are not terminal states but informative transitions in the research trajectory. By abstracting structural lessons from imperfect executions, the agent approximates the human scientific process of hypothesis revision under incomplete evidence.

F Experiment Summary Example

Listing 6: Example of Experiment Summary for Cooperative Driving problem.

```
>>>[ExperimentAgent] Final summary after 5 experiments:
The experiment series consistently demonstrates a catastrophic and persistent
deadlock issue that renders the current 'driving_actions' implementation
fundamentally flawed. Across all experiments (4-6), traffic rapidly deteriorates
into complete gridlock by step 150, with all vehicles (61-62) becoming completely
stationary (0 m/s average speed) and remaining so for the entire simulation
duration. This pattern is not transient but systematic, indicating a core
algorithmic failure rather than parameter tuning issues.

**Key Findings**: The root cause is a complete lack of cooperative lane-changing
coordination. Vehicles consistently desire lane changes (45-46 out of 62 vehicles)
but are universally blocked due to insufficient space in target lanes. This
creates a cascading deadlock where no vehicle can move, leading to 0.0m minimum
gaps between stopped vehicles and complete traffic paralysis. The algorithm fails
to implement any deadlock detection or resolution mechanisms.

**Performance Trends**: All experiments show identical catastrophic performance
metrics: average speed of 0.2 m/s (near-zero efficiency), 1 collision, 150
critical TTC events (severe safety violations), 5 teleports, and high speed
variance. The score remains consistently at -66.98 across all experiments,
confirming that no meaningful improvement occurred through the refinement process.

**Parameter Insights**: The experiments reveal that the issue is not related to
parameter tuning but to fundamental algorithmic design. The problem persists
regardless of monitoring callback enhancements, indicating that better observation
alone cannot solve the underlying coordination failure.

**Documented Failures**: The current approach of processing individual vehicle lane
change requests without global coordination consistently fails. The algorithm
treats lane changes as independent decisions rather than requiring cooperative
negotiation between vehicles. This leads to situations where every vehicle is
blocked by others, creating an unsolvable circular dependency.

**Useful Techniques**: While the current implementation fails, the monitoring
callbacks developed in experiments 4-6 provide valuable diagnostic capabilities
for detecting gridlock formation, analyzing vehicle behavioral patterns, and
measuring lane change blocking rates. These diagnostic tools should be retained
for future development.

**Unresolved Issues**: The primary unresolved issue is the lack of a cooperative
lane-changing protocol that can resolve conflicting requests and prevent deadlock
formation. The algorithm needs fundamental redesign to include: (1) deadlock
detection mechanisms, (2) coordinated lane change scheduling, (3) temporary
yielding strategies to break circular dependencies, and (4) dynamic lane
assignment based on route continuation needs.

**Actionable Recommendations**: Future research should focus on implementing a
centralized or distributed coordination mechanism for lane changes. Specific
directions include: (1) Develop a priority-based lane change system that considers
route continuation needs and vehicle types, (2) Implement deadlock detection that
```

triggers emergency yielding protocols when gridlock is imminent, (3) Design a cooperative gap creation mechanism where vehicles temporarily adjust speed to create space for essential lane changes, (4) Introduce memory of previous actions to prevent oscillatory behavior, and (5) Prioritize safety over efficiency during high-congestion scenarios by temporarily suppressing non-essential lane changes. The current approach must be abandoned in favor of a fundamentally cooperative framework that treats the vehicle fleet as a coordinated system rather than independent agents.

The experiment series reveals a clear progression from identifying a fundamental safety flaw to attempting comprehensive algorithmic fixes, with mixed results.

****Key Findings**:** Experiments 1-7 consistently showed identical poor performance (58 collisions, 246 critical TTC events, score ~ -1436.98) due to an overly aggressive lane-changing strategy that prioritized efficiency over safety. The root cause was definitively identified through callback monitoring showing persistent high lane-changing activity (2-7 vehicles per checkpoint). Experiment 7 attempted a complete safety-first rewrite with proper car-following logic, conservative lane-changing criteria, and cooldown periods, but Experiment 8 failed with a timeout, indicating severe performance issues in the new implementation.

****Performance Trends**:** The first seven experiments showed zero improvement despite parameter exploration, confirming that the original algorithm had a systemic safety flaw rather than tunable parameters. The attempt to fix this with a comprehensive rewrite introduced computational inefficiencies that caused simulation timeouts.

****Persistent Challenges**:** The core issue of balancing lane-changing desires with safety constraints couldn't be resolved through simple parameter tuning. The timeout in Experiment 8 suggests that complex vehicle grouping and sorting operations may be computationally expensive at scale.

****Successful Strategies**:** The identification methodology using callbacks to monitor lane-changing activity proved highly effective for root cause analysis. The conceptual approach of safety-first lane changing with longitudinal gap requirements (15m minimum) and cooldown periods is theoretically sound.

****Documented Failures**:** Simply implementing complex cooperative logic without considering computational efficiency led to simulation failure. Using position as a proxy for time steps in cooldown calculations may be unreliable.

****Useful Techniques**:** Vehicle grouping by lane and sorting by position enables proper leader-follower relationships. Conservative acceleration limits (max_accel=1.2, comfortable_braking=2.5) and increased safe distances (5.0 + speed*0.8) provide better safety margins.

****Unresolved Issues**:** Computational efficiency of the algorithm needs significant optimization to prevent timeouts. The lane-changing safety logic may still be too restrictive or inefficiently implemented.

****Actionable Recommendations**:** 1) Simplify the algorithm by removing unnecessary complexity while maintaining safety principles, 2) Optimize data structures and avoid expensive operations like repeated sorting, 3) Use more reliable time tracking instead of position-based approximations, 4) Implement incremental improvements rather than complete rewrites to isolate what works, 5) Add explicit collision avoidance as the highest priority override to all other behaviors, 6) Consider using the potential_target_lanes information more effectively to make informed lane decisions rather than just checking immediate neighbors.

G Solution Reversion Example

During iterative experimentation, it is common for performance to deteriorate after a series of modifications. In such cases, reverting to a previously better-performing solution before terminating the experiment can be beneficial. OR-Agent supports two forms of solution reversion: *soft reversion* and *hard reversion*.

G.1 Soft Reversion via Prompt Engineering

Soft reversion is implemented through explicit prompting. The Experiment Agent is instructed that, if performance degrades and the experiment is approaching termination, it may revert to a previous

version of the solution code or parameter configuration. For example:

Important: this is the final attempt to improve the solution. You may revert to a previous version of the solution code or parameter configuration if the latest attempt is unsatisfactory. Ensure the final solution code is executable.

In many cases, large language models successfully perform this reasoning-driven rollback. An illustrative example from the OP-ACO problem is shown below.

Listing 7: Soft reversion example: regression detection and rollback reasoning.

```
>[Evaluator] Metrics extracted for solution (id): lead1_round1_count0_id21
metrics: {50: 11.227, 100: 14.992}
features: (11, 14)
scores: 13.109

>[ExperimentAgent] Thinking: Looking at the latest experiment results and comparing
with previous ones:
Previous experiment (#2):
- Problem size 50: 11.234
- Problem size 100: 15.199
- Overall score: 13.217

Current experiment (#3):
- Problem size 50: 11.227 (slight decrease of ~0.007)
- Problem size 100: 14.992 (decrease of ~0.207)
- Overall score: 13.109 (decrease of ~0.108)

I can observe a clear pattern: my recent changes led to a deterioration in
performance. The modifications I made were:
1. Reduced clustering radius from 0.2*max_dist to 0.15*max_dist
2. Changed high_value_threshold back from 70th to 75th percentile
3. Adjusted weights to 0.6/0.25/0.15 (more emphasis on basic heuristic)
4. Removed some of the depot-specific handling logic

The decrease in performance suggests that my changes were not beneficial. The
previous configuration with 0.5/0.3/0.2 weights and the more sophisticated depot
handling was actually better.
Looking at the overall trend:
- Experiment 1: Score 11.860
- Experiment 2: Score 13.217 (significant improvement)
- Experiment 3: Score 13.109 (regression)

Since this is a regression from the best result, I should revert some changes and try
a different approach. Let me go back to the better performing configuration but
try some other optimizations like adding a more sophisticated budget-aware
component that considers not just the direct return to depot but also potential
for visiting multiple nodes before returning.
```

In this example, the agent correctly identifies performance regression and reasons about which modifications likely caused the degradation. It then decides to restore a previous configuration before exploring alternative improvements.

G.2 Hard Reversion via System-Level Control

Despite its effectiveness, soft reversion is not always reliable. In certain cases, the language model fails to restore the best-performing configuration accurately. To ensure robustness, we therefore implemented a *hard reversion* mechanism. Hard reversion is performed at the system level: the best solution encountered during the experiment is manually selected and restored before final summarization. The Experiment Agent is informed of this reversion so that its concluding reflection incorporates the corrected trajectory. The following example from the cooperative driving problem illustrates hard reversion combined with experiment summarization.

Listing 8: Hard reversion combined with final experiment summary (Driving Problem).

```
>[Evaluator] Metrics extracted for solution (id): lead1_round1_count0_id21
metrics: {'avg_speed': 0.290, 'emergencyStops': 0.0, 'speed_variance': 2.41,
'collisions': 0.5, 'emergencyBraking': 0.0, 'critical_ttc_count': 144.0,
'teleports': 5.5}
```

```
features: (0, 0, 3)
scores: -60.246
```

```
>[ExperimentAgent] Revert back to previous code version | score reverted from -60.246
to -13.512
```

```
>[ExperimentAgent] Final summary after 5 experiments:
```

The experiment series aimed to optimize a hierarchical cooperative negotiation system for cooperative driving. Over six experiments, the solution evolved from an initial score of -35.132 to a best score of -13.511 (experiment #5), slightly outperforming the parent solution (-14.068) but revealing fundamental inefficiencies.

Key findings include: 1) Average speeds remained critically low (0.29-0.56 m/s vs. target 13.89 m/s), indicating severe traffic flow impairment; 2) Safety metrics fluctuated, with collisions introduced and partially mitigated through parameter tuning; 3) Teleports persisted (3.5-5.5), signaling deadlock-like conditions where vehicles stalled; 4) Excessive lane change attempts (13-28 per step) disrupted stability. Performance trends showed initial improvement through parameter adjustments (e.g., reducing time headway, increasing lane change cooldown), but experiment #6 regressed to -60.245, suggesting over-tuning or instability.

Persistent issues include the algorithm’s over-complexity, which may hinder flow compared to simpler alternatives like the parent’s predictive platoon formation. Successful strategies included state persistence for consistency, predictive safety checks, limiting simultaneous lane changes, and hysteresis in decision-making. However, unresolved limitations require fundamental changes: the negotiation framework introduces latency, cooperative adjustments often slow vehicles excessively, and low-speed handling fails to prevent teleports.

Recommendations for future research: simplify the algorithm toward light-weight cooperation; prioritize speed maintenance with aggressive IDM tuning and deadlock resolution; reduce lane change frequency through stricter urgency thresholds; and implement proactive flow maintenance rather than reactive negotiation. This analysis underscores that incremental parameter tuning is insufficient-structural redesign focusing on efficiency and simplicity is needed to achieve significant score improvements.

This example demonstrates several important properties:

- **Regression detection:** The agent identifies deterioration relative to historical best performance.
- **Rollback capability:** Hard reversion guarantees that final results reflect the best discovered solution.
- **Failure-aware summarization:** Even after rollback, the agent synthesizes insights from unsuccessful trials.
- **Structural diagnosis:** The final reflection often reveals that parameter tuning alone is insufficient and that architectural redesign is necessary.

G.3 Discussion

Solution reversion plays a critical role in preventing performance collapse during deep experimentation. Soft reversion enables autonomous reasoning-based rollback, while hard reversion provides system-level safety guarantees. More importantly, reversion is not merely a defensive mechanism. It allows OR-Agent to preserve strong baselines while continuing exploratory modifications, closely mirroring scientific practice: researchers frequently revert to a stable configuration before pursuing a new direction. This combination of rollback and structured reflection ensures that exploration does not irreversibly destroy accumulated progress.

H Long-Term Reflection Example

This appendix provides an example of a long-term reflection generated during the MKP-ACO experiments. Long-term reflection accumulates cross-experiment observations, identifies recurring patterns and persistent challenges, and proposes structured directions for future improvement. Although OR-Agent does not enforce an explicitly structured storage format for long-term reflections, the language

model typically produces well-organized analytical summaries. These reflections typically include: (1) empirical pattern synthesis, (2) diagnosis of architectural limitations, (3) extraction of robust design principles, and (4) forward-looking research hypotheses. An illustrative example is shown below.

Listing 9: Example of long-term reflection in MKP_ACO experiments.

```
>[LeadAgent] Long-term reflection updated to:
Recent experiments strongly confirm that bottleneck-aware heuristics-particularly the
minimum prize-to-weight ratio across dimensions or prize/max_weight-are
consistently superior for MKP in ACO, outperforming harmonic/geometric means and
sum-based aggregations. Simplicity enhances robustness: complex adaptive schemes,
interaction models, and lookahead mechanisms often degrade performance due to
noise or misaligned signals. The harmonic mean shows moderate promise but is less
effective than min-ratio approaches, which naturally enforce feasibility by
prioritizing worst-case constraint efficiency.

Persistent challenges include the static nature of heuristics, inability to adapt to
residual capacities during construction, and handling scale heterogeneity across
problem sizes. Successful principles center on clear, interpretable efficiency
metrics with epsilon-stabilized ratios.

Future work should prioritize lightweight dynamic heuristics incorporating real-time
capacity usage, instance-specific normalization, and hybrid designs that blend
bottleneck focus with total resource awareness-potentially via learned or
correlation-informed weights-while avoiding over-engineering. Future work must
prioritize truly dynamic heuristics that recompute item desirability using
real-time knapsack states, develop capacity-pressure-based conflict measures, and
explore instance-adaptive blending of efficiency and compatibility-while resisting
over-engineering. The path forward lies in minimal, bottleneck-grounded heuristics
enhanced by online feasibility awareness.
```

This example highlights several important characteristics of long-term reflection in OR-Agent:

- **Pattern Consolidation:** The reflection synthesizes evidence across multiple experiments, identifying consistent empirical regularities (e.g., the dominance of bottleneck-aware ratios).
- **Failure Diagnosis:** It explicitly recognizes the limitations of static heuristics and over-engineered enhancements.
- **Principle Extraction:** General design principles (simplicity, interpretability, bottleneck dominance) are distilled from empirical outcomes.
- **Future Direction Articulation:** The reflection proposes concrete research hypotheses, such as incorporating dynamic capacity awareness.

Importantly, the reflection moves beyond incremental parameter tuning and instead reasons at the architectural level. This demonstrates that long-term reflection functions not merely as memory accumulation, but as a mechanism for meta-level abstraction and strategic redirection of the research trajectory.

I Example Trajectory of OR-Agent on Traveling Salesman Problem

The Traveling Salesman Problem (TSP) is a classical combinatorial optimization problem that seeks the shortest possible tour visiting each city exactly once and returning to the starting city. Due to its NP-hard nature, TSP serves as a standard benchmark for evaluating heuristic design, reinforcement learning policies, and combinatorial search methods. In this case study, TSP is solved using the POMO (Policy Optimization with Multiple Optima) (?) reinforcement learning framework. Rather than training the policy end-to-end, the objective of OR-Agent is to design a task-specific **heuristics** function that generates an attention bias matrix for the POMO neural network. The quality of a candidate solution is evaluated by the average tour distance across test instances of different sizes (200, 500, and 1000 cities).

This section presents an example trajectory of OR-Agent on the TSP_POMO problem, illustrating how the framework performs both structural improvement and parameter refinement over the course

of experimentation. A representative trajectory is shown in Listing 10. In each iteration, the evaluator first assesses the current candidate and returns the corresponding results; the experiment agent then analyzes the feedback, decides on the next modification, and launches the subsequent experiment. Listing 10 presents three experiment steps, each of which is explained in detail below.

Experiment #1: Timeout Diagnosis and Algorithm Simplification. The initial candidate solution introduced a geometric graph ensemble combining Gabriel graph, Relative Neighborhood Graph (RNG), and k-nearest neighbor graph (k-NNG). However, evaluation resulted in a 300-second timeout for large-scale instances. The Experiment Agent identified that nested triple loops in the Gabriel and RNG constructions resulted in $O(n^3)$ complexity, rendering the approach computationally infeasible for large TSP instances. The agent compared this solution to its parent solution (Node 54), which achieved stable performance. Consequently, the entire `heuristics` function was rewritten (via a full merge-conflict diff update) to remove cubic geometric constructions while preserving effective structural components. For relatively simple OR problems such as TSP, it was observed that the Experiment Agent frequently performs complete functional rewrites rather than minor parameter adjustments—highlighting a deviation from the intended “small-step refinement” assumption. Re-evaluation yielded: {200 : 11.112, 500 : 20.929, 1000 : 31.413}, score = 21.151. The timeout issue was resolved while preserving performance.

Experiment #2: Refinement of k-NNG. The Experiment Agent next refined the k-NNG computation, making k adaptive to problem size and adjusting bonus weights. This revision targeted large-scale connectivity, hypothesizing that larger instances require denser neighborhood structures. The updated evaluation results were: {200 : 11.114, 500 : 20.870, 1000 : 31.288}, score = 21.091. Although the agent described the 200-city result (11.114 vs. 11.112) as a “slight improvement,” this was in fact a minor degradation. This illustrates a known limitation of LLM reasoning over floating-point comparisons. Nevertheless, the overall score improved due to gains in larger instances, demonstrating effective large-scale bias tuning.

Experiment #3: Parameter Tuning. Subsequent revisions focused on fine-tuning MST bonus coefficients and size-dependent scaling factors. These adjustments were minor relative to the structural changes in earlier steps, reflecting a shift from architectural revision toward local optimization.

Listing 10: Example of experiment trajectory on TSP_POMO problem.

```
>[Evaluator] Evaluating solution: lead1_round12_count19_id71
Error: no end marker found in the result. Timed out after 300 seconds.

>[ExperimentAgent] Thinking: Looking at the latest experiment result, I can see that
the execution timed out after approximately 300 seconds. This indicates that the
current implementation of the 'heuristics' function is computationally expensive
and inefficient. Let me analyze the current solution:
1. The function implements a geometric graph ensemble combining Gabriel graph, RNG,
and k-NNG
2. For the Gabriel graph, there's a nested loop structure: 'for i in range(n)' and
'for j in range(i + 1, n)' and then another 'for k in range(n)' - this creates
 $O(n^3)$  complexity
3. For the RNG, there's similar  $O(n^3)$  complexity with nested loops
4. Additionally, there are eigenvalue computations for the Gram matrix which are
expensive for large matrices
5. The adaptive neighborhood computation also adds complexity

Comparing to the parent solution (#54) which achieved a score of 21.151, that
solution had:
- Adaptive local neighborhoods with rank-decayed  $-\log(d)/\text{rank}$  formulation
- MST computation (which has  $O(E \log V)$  complexity, manageable)
- k-NNG computation (which is  $O(n*k)$  where  $k$  is small)
- No  $O(n^3)$  nested loops

The parent solution avoided the expensive  $O(n^3)$  computations that are causing the
timeout. The current solution's Gabriel graph and RNG implementations are
extremely expensive for large TSP instances (200, 500, 1000 cities). The timeout
suggests that for larger instances (especially 500 and 1000 city problems), the
 $O(n^3)$  computations are taking too long. I need to simplify the approach to make
it computationally feasible while retaining the beneficial aspects. I should
modify the solution to:
1. Keep the adaptive local neighborhoods (from parent solution) which were effective
2. Keep the MST computation (from parent solution) which was effective
3. Keep a simpler k-NNG computation (from parent solution)
```

```

4. Remove the expensive  $O(n^3)$  Gabriel and RNG computations
5. Simplify the instance-level metrics computation to avoid expensive eigenvalue
   calculations

>[ExperimentAgent] Action after experiment 1: update code
<<<<<< SEARCH
def heuristics(distance_matrix: torch.Tensor) -> torch.Tensor:
...
=====
...(whole 'heuristic' function is updated)...
>>>>>>

>[Evaluator] Evaluating solution: lead1_round12_count19_id71
Metrics: {200: 11.112, 500: 20.929, 1000: 31.413}; features: (11, 20, 31); scores:
21.151.

>[ExperimentAgent] Thinking: Looking at the latest experiment result, I can see that
the solution achieved a score of 21.151 with metrics {200: 11.112, 500: 20.929,
1000: 31.413}. This is almost identical to the parent solution #54's performance,
which means my simplification successfully preserved the performance while fixing
the timeout issue. This approach is much more computationally efficient and
achieves the same performance as the parent solution. However, I need to consider
if there are further improvements possible. Looking back at the parent solution
analysis, the experiments showed that:
- Complex dynamic approaches often underperformed compared to simpler approaches
- The combination of negative distance initialization + adaptive local neighborhoods
  + MST + k-NNG was effective
- Scale robustness remained challenging

One possibility is to refine the bonus calculation or improve the adaptive
neighborhood approach. Looking at the current solution, I notice that the adaptive
neighborhood approach uses a fixed  $k=15$  for density estimation, but this might not
be optimal for all problem sizes. Also, the MST and k-NNG bonuses could be
fine-tuned. Let me try to enhance the solution by:
1. The  $k$  values ( $k_{\text{for\_density}}=15$ ,  $k_{\text{nn}}=6$ ) could be made more adaptive
2. The bonus weights (0.5 for MST, 0.3 for k-NNG) could be tuned
3. Maybe we could add another simple geometric structure that's efficient
Let me try to increase the effectiveness of the k-NNG by increasing  $k$  slightly for
larger problems, since for larger problems, we might need more connectivity.

>[ExperimentAgent] action after experiment 2: update code
<<<<<< SEARCH
    # Compute k-Nearest Neighbor Graph (replacing expensive Gabriel/RNG with
    # efficient k-NNG)
...
=====
    ...("Compute k-Nearest Neighbor Graph" code section is revised)...
>>>>>>

>[Evaluator] Evaluating solution: lead1_round12_count19_id71
Metrics: {200: 11.114, 500: 20.870, 1000: 31.288}; features: (11, 20, 31); scores:
21.091.

>[ExperimentAgent] Thinking: Looking at the latest experiment results, I can see the
following:
Previous best score (parent solution #54): 21.15. Current experiment score: 21.091.
The metrics are:
- 200 cities: 11.114 (was 11.112) - slight improvement
- 500 cities: 20.870 (was 20.929) - improvement
- 1000 cities: 31.288 (was 31.413) - significant improvement

This shows that my modification was successful! The score improved from 21.151 to
21.091, which represents a meaningful improvement. The changes I made were:
1. Made  $k_{\text{NN}}$  adaptive based on problem size (6->8->10 for different sizes)
2. Increased the  $k_{\text{NN}}$  bonus weight from 0.3 to 0.35
3. Enhanced the size-based adjustment factors

The improvements are particularly noticeable for larger problem sizes (500 and 1000
cities), which aligns with the intuition that larger problems need more
connectivity.

```

However, I should consider if there are additional improvements possible. Let me think about what else could be optimized:

1. The adaptive neighborhood component could be refined - currently using $k_{\text{for_density}} = \min(15, n-1)$
2. The MST bonus parameters could be fine-tuned further
3. Maybe there are other lightweight geometric structures I could add
4. The adaptive local neighborhood parameters ($-\log(d)/\text{rank}$ formulation) could be optimized

Let me try to improve the adaptive neighborhood component by adjusting the $k_{\text{for_density}}$ value and potentially refining the ranking approach:

Looking back at the parent solution analysis, the best performing configurations had a good balance between local adaptive neighborhoods, MST, and k-NNG. My current solution improved the k-NNG part, but maybe I can further improve the MST bonus or the adaptive neighborhood parameters.

```
>[ExperimentAgent] action after experiment 3: update code
<<<<<<< SEARCH
    # Calculate adaptive bonus based on average distance (similar to parent
    # solution)
    mst_bonus = 0.5 * (1.0 + 1.0 / (1.0 + avg_distance_global))

    # Also incorporate problem size factor to enhance performance on larger
    # instances
    if n > 500:
        mst_bonus *= 1.2 # Slight enhancement for large problems
    elif n < 200:
        mst_bonus *= 0.8 # Slight reduction for small problems
=====
    # Calculate adaptive bonus based on average distance (enhanced from parent
    # solution)
    mst_bonus = 0.55 * (1.0 + 1.0 / (1.0 + avg_distance_global))

    # Also incorporate problem size factor to enhance performance on larger
    # instances
    if n > 500:
        mst_bonus *= 1.25 # Enhanced boost for large problems
    elif n < 200:
        mst_bonus *= 0.85 # Slight reduction for small problems
>>>>>>> REPLACE
...
```

Research Tree Dynamics The complete research tree for this round is shown in Figure 3. The round was initialized from an elite solution (Node 51) as the root. Five child nodes (Nodes 52–56) were generated, representing distinct research directions forming a comprehensive exploration plan. OR-Agent does not require immediate improvement at depth 1. The maximum tree depth was set to 3. A child node represents a refinement or extension of its parent’s idea. For example, Node 57 inherits the core concept introduced in Node 53 and achieves a modest performance improvement. Except for the root, child nodes are retained only if they achieve performance improvement relative to their parent. After maximum number of failed expanding attempts, a non-improving leaf node is marked as “approximate local optimum” and no longer expanded. All approximate local optimum nodes are returned to the solution database; alternative filtering mechanisms may be considered in future implementations. The final research tree contained 14 nodes, of which 7 were expanded and 7 were terminal. The best solution of the round improved the database record from 21.12 to 20.98. This example demonstrates several important behaviors of OR-Agent:

1. Progressive revision through staged refinement: Individual experimental attempts could yield temporary degradations in performance. Performance improvements typically emerged through multiple experimental iterations rather than single-step updates.
2. Systematic branching: Multiple independent directions were explored concurrently, increasing the probability of discovering non-obvious improvements.
3. Depth-based discovery: The best solution did not appear among immediate children but required deeper refinement.

Overall, this case study illustrates how OR-Agent integrates evolutionary initialization, tree-structured exploration, and iterative refinement to progressively improve heuristic quality.

J Example Trajectory of OR-Agent on Cooperative Driving Problem

This case study considers a cooperative driving control problem implemented in the SUMO (Simulation of Urban MObility) traffic simulator (?). The objective is to design a Python function named `driving_actions`, which takes road segment information and real-time vehicle states as inputs and outputs driving actions (including acceleration and lane-change decisions) for all vehicles on a given road segment. The resulting algorithm is evaluated using multi-objective criteria including safety, speed efficiency, and traffic smoothness, and the overall goal is to maximize a scalar performance score derived from these metrics. Unlike the Traveling Salesman Problem, this task operates within a complex simulation environment and presents additional challenges that substantially increase its difficulty.

Diverse and Complementary Idea Generation At the beginning of a research round, OR-Agent generated multiple complementary high-level strategies, forming a coherent research plan rather than isolated mutations. Representative examples are shown in Listing 11. These ideas demonstrate significant structural diversity while addressing different dimensions of the coordination problem (conflict anticipation, dynamic routing, cooperative grouping, and adaptive safety modeling). This joint generation phase reflects OR-Agent’s ability to explore heterogeneous research directions.

Listing 11: Example of coordinated idea generation for cooperative driving problem.

1. Develop a predictive conflict-resolution framework that uses short-term trajectory forecasting to proactively resolve lane-change conflicts before they arise, replacing reactive swap detection with anticipatory coordination based on vehicle intentions and kinematic constraints.
2. Implement an adaptive urgency-aware lane assignment system that dynamically re-evaluates target lanes using real-time congestion metrics from `'potential_target_lanes'`, prioritizing lanes that balance route continuation with current traffic flow efficiency rather than static best-lane offsets.
3. Design a hierarchical cooperative driving protocol where vehicles form temporary coordination groups based on shared route segments and spatial proximity, enabling synchronized acceleration/deceleration patterns that reduce speed variance and improve platoon stability beyond simple pairwise interactions.
4. Introduce a machine learning-based gap acceptance model trained on historical safe lane-change scenarios to replace fixed gap thresholds, allowing the algorithm to adapt gap requirements based on contextual factors like vehicle types, speeds, and traffic density while maintaining safety guarantees.

Environment Exploration via Callbacks In complex environments such as cooperative driving simulation, poor performance cannot be adequately explained by final scores alone. Aggregate metrics (e.g., collisions, average speed) indicate *what* failed, but not *why*. By introducing targeted callbacks, OR-Agent collects fine-grained behavioral signals—such as lane-change attempts, vehicle stagnation, congestion buildup, and coordination failures—allowing iterative scientific-style diagnosis. Listing 12 shows an early-stage evaluation prior to callback refinement.

Listing 12: Initial evaluation output before callback-based environment probing.

```
>[Evaluator] Evaluating solution: lead1_round1_count0_id21
metrics: {'teleports': 3.5, 'avg_speed': 0.775, 'speed_variance': 6.17,
          'collisions': 30.0, 'emergencyBraking': 0.5,
          'emergencyStops': 0.0, 'critical_ttc_count': 297.5}
features: (0, 0, 2)
scores: -806.773
```

The Experiment Agent performs structured reasoning over these signals, identifying potential causes such as overly aggressive cooperative gap creation, insufficient safety margins in lane-change logic, and unstable acceleration behavior. However, these hypotheses require deeper environmental visibility. To move from symptom observation to mechanism diagnosis, the agent introduces monitoring callbacks

```

>[LeadAgent] Final research tree for research round #12:
Format: Node <ID> (<score>): "<idea>"
Legend:
  ⊕ = solution expanded with improved children solution(s)
  ✓ = terminal solution (local optimum, no improvement found)
PS: Only root node may contain more than one solutions.
=====
Node [51 (21.12)]: ["Integrate Delaunay triangulation as a complementary global
structure to MST..."]
  +— ✓ Node 52 (21.07): "Develop a scale-invariant normalization scheme for distance-
based heuristics..."
  +— ⊕ Node 53 (21.05): "Replace the KNN-based Delaunay approximation with a mutual
Gabriel graph construction..."
    |   └─ ✓ Node 57 (21.03): "Develop a hybrid geometric-graph heuristic that
combines the mutual Gabriel graph with a k-Nearest Neighbor Graph (k-NNG) using
adaptive edge fusion..."
  +— ⊕ Node 54 (21.15): "Introduce dynamic, instance-aware bonus allocation by
computing structural complexity metrics (e.g., coefficient of variation in local
density, MST total weight relative to random baseline) and using them to modulate MST
and Delaunay-like bonuses through learned or rule-based functions..."
    |   └─ ✓ Node 71 (21.09): "Develop a coordinate-free geometric graph ensemble
that combines multiple sparse proximity graphs..."
    |   └─ ⊕ Node 72 (21.10): "Replace hand-crafted bonus scaling with a lightweight,
offline-trained meta-heuristic predictor that maps structural features of the distance
matrix to optimal attention bias coefficients.."
    |       └─ ✓ Node 75 (21.09): "Develop a dynamic k-selection mechanism for
the Delaunay-like k-NN graph that adapts k per node based on local density metrics..."
  +— ✓ Node 55 (21.10): "Implement a lightweight coordinate reconstruction module using
differentiable classical MDS (Multidimensional Scaling) as a pre-processing step..."
  +— ⊕ Node 56 (21.12): "Reformulate the local neighborhood heuristic to use entropy-
weighted ranking instead of uniform rank decay..."
    +— ⊕ Node 65 (21.02): "Develop a hybrid attention bias scheme that combines
coordinate-free geometric graphs (MST, mutual Gabriel graph, and k-NNG) with a
lightweight differentiable MDS module to reconstruct 2D coordinates..."
    |   +— ✓ Node 67 (20.98): "Develop a learned, lightweight fusion
mechanism that dynamically weights the contributions of local neighborhood
heuristics..."
    +— ⊕ Node 66 (21.12): "Design a dynamic, instance-adaptive normalization and
bonus allocation mechanism that replaces fixed scaling factors (e.g., MST
bonus = 0.5) with data-driven modulation based on intrinsic problem
characteristics..."
      +— ✓ Node 70 (21.09): "Introduce a tour-construction-aware dynamic
biasing mechanism that simulates partial greedy path extensions during
heuristic computation to identify 'critical edges'-those that
frequently appear in early-stage greedy tours..."
=====
Total expanded solutions: 7 | Total pending leaves: 0 | Total terminal leaves: 7
>[LeadAgent] number of solutions to return: 7 (IDs: 52, 55, 57, 67, 70, 71, 75)
>[Database] Best score updated to 20.98

```

Figure 3: Research tree example for one research round on TPS_POMO problem.

that log intermediate state statistics, as shown by Listing 13. The resulting logs has more than 120 lines for each test case hence truncated as shown by Listing 14. Still, it confirms that the failure is not purely due to unsafe gaps, but rather a systemic interaction between lane-change instability and flow breakdown.

Listing 13: First callback instrumentation for monitoring congestion and lane-change behavior.

```
class Callbacks:
    def __init__(self):
        self.step_counter = 0
        self.dangerous_gaps = 0
        self.low_speeds = 0
        self.lane_change_attempts = 0

    def on_step_end(self, **kwargs):
        """Monitor safety metrics and log critical events."""
        step = kwargs["step"]
        vehicles_info = kwargs["vehicles_info"]
        edge_info = kwargs["edge_info"]

        self.step_counter += 1

        # Log every 100 steps
        if step % 100 == 0:
            print(f"Step{step}: Monitoring {len(vehicles_info)} vehicles on edge {edge_info['edge_id']}")

        # Check for dangerous situations
        dangerous_count = 0
        low_speed_count = 0

        for vehicle in vehicles_info:
            # Check for very low speeds (potential congestion)
            if vehicle["speed"] < 1.0:
                low_speed_count += 1

            # Check for dangerous gaps (simplified - would need leader info)
            # This is a placeholder for more detailed analysis

        if dangerous_count > 0 and step % 10 == 0:
            print(f"Step{step}: {dangerous_count} vehicles in dangerous gaps")

        if low_speed_count > len(vehicles_info) * 0.5 and step % 50 == 0:
            print(f"Step{step}: Warning - {low_speed_count}/{len(vehicles_info)} vehicles with speed < 1.0 m/s")

        # Track lane change attempts
        lane_changes = sum(1 for v in vehicles_info if v.get('wants_left', False) or v.get('wants_right', False))
        if lane_changes > 0 and step % 20 == 0:
            print(f"Step{step}: {lane_changes} vehicles attempting lane changes")
```

Listing 14: SUMO output after first callbacks update.

```
>[Evaluator] Evaluating solution: lead1_round1_count0_id21
...(Omitted for brevity)...
Step 100.0: Monitoring 20 vehicles on edge E0
Step 100.0: 4 vehicles attempting lane changes
Step 120.0: 6 vehicles attempting lane changes
Step 140.0: 3 vehicles attempting lane changes
Step 150.0: Warning - 24/30 vehicles with speed < 1.0 m/s
...
[Output truncated: 124 total lines, showing first 50 and last 50 lines]
...
Step 200.0: 29 vehicles attempting lane changes
Step 220.0: 29 vehicles attempting lane changes
Step 240.0: 29 vehicles attempting lane changes
Step 250.0: Warning - 60/60 vehicles with speed < 1.0 m/s
...(Omitted for brevity)...

metrics: {'teleports': 4.5, 'avg_speed': 0.42, 'emergencyStops': 0.0, 'collisions':
12.5, 'critical_ttc_count': 177.0, 'speed_variance': 2.88, 'emergencyBraking': 0.0}
```

```
features: (0, 0, 3)
scores: -353.158
```

After identifying large-scale stagnation, the Experiment Agent refines the callback instrumentation to measure acceleration behavior and lane-change effectiveness more explicitly, as shown by Listing 15. The new diagnostic output contains around 12 lines for each test case as shown by Listing 16. Rather than blindly optimizing scores, OR-Agent treats the environment as an object of investigation. Callbacks function as experimental instruments, enabling progressive refinement of hypotheses about system behavior. This structured exploration capability is essential for complex, interactive optimization problems. Check the public repository for more details (?).

Listing 15: Refined callback instrumentation focusing on flow and lane-change effectiveness.

```
class Callbacks:
    def __init__(self):
        self.step_counter = 0
        self.lane_change_attempts = 0
        self.lane_change_success = 0
        self.low_speed_vehicles = 0

    def on_step_end(self, **kwargs):
        """Monitor acceleration behavior and lane change effectiveness."""
        step = kwargs["step"]
        vehicles_info = kwargs["vehicles_info"]
        edge_info = kwargs["edge_info"]

        self.step_counter += 1

        # Calculate metrics
        total_vehicles = len(vehicles_info)
        low_speed_count = sum(1 for v in vehicles_info if v["speed"] < 2.0)
        lane_change_attempts = sum(1 for v in vehicles_info if v.get('wants_left',
            False) or v.get('wants_right', False))

        # Update accumulators
        self.low_speed_vehicles += low_speed_count
        self.lane_change_attempts += lane_change_attempts

        # Log every 50 steps
        if step % 50 == 0:
            avg_speed = sum(v["speed"] for v in vehicles_info) / max(total_vehicles,
                1)
            print(f"Step {step}: {total_vehicles} vehicles, avg speed {avg_speed:.2f} m/s, "
                f"{low_speed_count} vehicles < 2 m/s, {lane_change_attempts} lane change attempts")

            # Calculate and print lane change success rate if possible
            if self.lane_change_attempts > 0:
                success_rate = self.lane_change_success /
                    max(self.lane_change_attempts, 1)
                print(f"Lane change success rate: {success_rate:.1%}")

            # Reset accumulators for next window
            self.low_speed_vehicles = 0
            self.lane_change_attempts = 0
            self.lane_change_success = 0

        # Track successful lane changes (simplified: vehicles that changed lane since last step)
        # This would require storing previous lane info, but for simplicity we'll note attempts
        # For now, we'll assume a lane change is successful if the vehicle's wants_left/right is True
        # but this is not accurate. We'll leave as placeholder.
```

Listing 16: SUMO output after second callbacks update.

```
>[Evaluator] Evaluating solution: lead1_round1_count0_id21
Step 0.0: 0 vehicles, avg speed 0.00 m/s, 0 vehicles < 2 m/s, 0 lane change attempts
```

```

Step 50.0: 12 vehicles, avg speed 5.10 m/s, 6 vehicles < 2 m/s, 11 lane change
          attempts
          Lane change success rate: 0.0%
Step 100.0: 30 vehicles, avg speed 0.09 m/s, 30 vehicles < 2 m/s, 11 lane change
           attempts
           Lane change success rate: 0.0%
...(Omitted for brevity; round 10 lines for each test case)...

metrics: {'critical_ttc_count': 73.0, 'teleports': 4.5, 'avg_speed': 0.290,
          'emergencyStops': 0.0, 'speed_variance': 2.235, 'emergencyBraking': 0.0,
          'collisions': 0.5}
features: (0, 0, 3)
scores: -27.496

>[ExperimentAgent] Thinking: The latest experiment shows significant improvement in
          safety...(Omitted for brevity)...
>[ExperimentAgent] Action after experiment 3: update code
...

```

Solution Reversion During experimentation, performance occasionally regressed due to overly aggressive parameter modifications. In one instance, increasing deceleration severity reduced TTC violations but reintroduced collisions. The Experiment Agent autonomously reverted to a previous configuration, recognizing that safety improvements must not compromise stability (Listing 17). This soft reversion mechanism mirrors human debugging practice, where overfitting corrections are rolled back to restore balance.

Listing 17: Example of soft reversion during experiments for cooperative driving problem.

```

>[ExperimentAgent] Thinking: The latest experiment (Experiment #4) shows a
          significant regression from Experiment #3. The score dropped from 83.669 to
          62.611, a decrease of over 21 points...(check public repository for more
          details)...The issue seems to be that increasing the conflict severity factor made
          vehicles brake too hard to avoid predicted conflicts, creating new safety
          problems. We need to revert to a more conservative approach while still
          maintaining the improvements in critical TTC count.

```

Termination of Unpromising Directions In several cases, repeated experiments revealed persistent structural deficiencies, such as near-zero vehicle mobility, massive collision counts, and deadlock formation. When incremental parameter tuning failed to address architectural flaws, the Experiment Agent can terminate the direction and documented the need for a fundamental redesign (Listing 18).

Listing 18: Example of termination of unpromising directions during experiments for cooperative driving problem.

```

>[Evaluator] Evaluating solution: lead1_round0_count3_id3
metrics: {'emergencyStops': 0.0, 'emergencyBraking': 0.0, 'teleports': 8.0,
          'avg_speed': 0.32, 'critical_ttc_count': 1114.0, 'collisions': 5117.0,
          'speed_variance': 2.4}
features: (0, 0, 3)
scores: -128252.668
>[ExperimentAgent] Thinking: The latest experiment failed due to a syntax error,
          preventing proper evaluation. However, analyzing the pattern across all previous
          experiments reveals fundamental and persistent issues that cannot be resolved
          through simple parameter tuning or minor code adjustments...(check public
          repository for more details)...The problem requires a fundamental shift from
          individual optimization to coordinated multi-agent interaction with explicit
          protocols for conflict resolution, priority management, and deadlock prevention
>[ExperimentAgent] Action after experiment 4: terminate

```

Parameter Tuning in High-Dimensional Space The cooperative driving case also demonstrates fine-grained parameter tuning behavior. Adjustments to safety margins (e.g., `SAFE_TTC`, `MIN_GAP`) often revealed trade-offs between safety and efficiency. Increasing safety buffers reduced TTC violations but degraded traffic throughput, illustrating the inherent multi-objective tension (Listing 19).

Listing 19: Example of parameter tuning during experiments on cooperative driving problem.

```
>[ExperimentAgent] Thinking: The latest experiment shows continued safety
  issues...(check public repository for more details)...
>[ExperimentAgent] Action after experiment 2: update code
<<<<<<< SEARCH
  # Safety parameters
  SAFE_TTC = 2.5                # Safe time-to-collision threshold (seconds)
  MIN_GAP = 2.0                # Minimum gap for car following (meters); Note: gap
    does not include vehicle length
  LANE_CHANGE_GAP_BASE = 15.0 # Base gap required for lane changes (meters); Note:
    gap does not include vehicle length
  OFFSET_BUFFER = 2.0          # Safety buffer added to vehicle length for
    lane-changing (meters); Note: buffer does not include vehicle length
=====
  # Safety parameters
  SAFE_TTC = 3.0                # Safe time-to-collision threshold (seconds)
  MIN_GAP = 1.6                # Minimum gap for car following (meters); Note: gap
    does not include vehicle length
  LANE_CHANGE_GAP_BASE = 10.0 # Base gap required for lane changes (meters); Note:
    gap does not include vehicle length
  OFFSET_BUFFER = 1.6          # Safety buffer added to vehicle length for
    lane-changing (meters); Note: buffer does not include vehicle length
>>>>>>> REPLACE
```

K Code-Level Diversity of the Generated Algorithms

This appendix complements the idea-level comparison in Appendix C with a code-level analysis of the best solutions generated by OR-Agent and the baseline methods. We consider two representative problems: CVRP-LEHD, which uses a neural constructive solver, and TSP-CONSTRUCTIVE, which uses a purely heuristic constructive solver.

K.1 Methodology

We compare the five Python program solutions for each problem after removing the common module-level import scaffold. All generated function bodies and their helper functions are retained. Comments and docstrings are excluded. Identifiers are anonymized and literal values are replaced by their types, so the analysis is not affected by variable renaming, formatting, comments, or small changes to numerical constants.

We use two complementary similarity measures. First, each abstract syntax tree (AST) is represented as a multiset of normalized rooted subtree fingerprints of depth two. If $c_i(f)$ denotes the number of occurrences of fingerprint f in program i , normalized AST similarity is

$$S_{\text{AST}}(i, j) = \frac{\sum_f \min\{c_i(f), c_j(f)\}}{\sum_f \max\{c_i(f), c_j(f)\}}. \quad (1)$$

This multiset Jaccard measure preserves local tree structure while ignoring surface-level naming choices. The measure compares two programs by asking: “How many small AST patterns do these programs share, accounting for how often each pattern occurs?”

Second, we extract an operator vocabulary O_i from each program. It contains arithmetic, unary, Boolean, and comparison operators; control-flow constructs such as conditionals and loops; and invoked built-in, NumPy, and PyTorch operations. Calls to resolvable local helper functions are canonicalized. Operator overlap is the set Jaccard coefficient

$$S_{\text{op}}(i, j) = \frac{|O_i \cap O_j|}{|O_i \cup O_j|}. \quad (2)$$

Both measures range from zero (no measured overlap) to one (identical measured structure or vocabulary). We additionally report generated lines of code (LOC), AST node count, number of function definitions, a McCabe-style branch-point complexity, operator-vocabulary size, and the number of operators unique to one solution among the five solutions for that problem.

Table 1: Static code statistics after excluding the shared import scaffold. “Unique ops” counts operators that do not occur in any of the other four solutions for the same problem.

Problem	Generator	LOC	AST nodes	Functions	Branch complexity	Operator vocabulary	Unique ops.
CVRP-LEHD	FunSearch	123	834	1	2	33	6
	AEL	123	788	1	3	29	5
	EoH	53	288	1	2	16	2
	ReEvo	39	173	1	1	8	1
	OR-Agent	173	1183	1	10	45	21
TSP-CONSTRUCTIVE	FunSearch	641	4269	4	145	38	12
	AEL	140	663	3	19	26	2
	EoH	63	280	1	7	18	0
	ReEvo	138	828	3	15	26	1
	OR-Agent	131	536	4	11	21	2

K.2 CVRP-LEHD: Strong Structural and Mechanistic Diversity

Table 2 reports all pairwise similarities. Across the ten pairs, mean AST similarity is 0.322, with a range of 0.120–0.614; mean operator overlap is 0.313, with a range of 0.152–0.476. Thus, an average pair differs in approximately 68% of its normalized AST-subtree mass and 69% of its operator vocabulary.

Table 2: Pairwise similarity for CVRP-LEHD. Each entry is $S_{\text{AST}}/S_{\text{op}}$.

	FunSearch	AEL	EoH	ReEvo	OR-Agent
FS	1.000/1.000	0.614/0.476	0.272/0.324	0.166/0.171	0.474/0.368
AEL		1.000/1.000	0.309/0.452	0.186/0.233	0.446/0.298
EoH			1.000/1.000	0.423/0.412	0.208/0.245
ReEvo				1.000/1.000	0.120/0.152
OR-A					1.000/1.000

The programs also implement different routing signals:

- **FunSearch** builds a proximity-and-density heuristic. It combines inverse distance, Gaussian-like demand density, a symmetric top- k neighbor mask, depot bonuses, capacity penalties, demand complementarity, and a quantile-based density bonus.
- **AEL** constructs a broad linear ensemble of hand-designed terms: inverse distance, Clarke–Wright savings, capacity utilization, demand-to-distance ratio, spatial preference, demand similarity, local density, top- k neighbors, demand complementarity, depot attraction, and route-continuation preference.
- **EoH** is substantially more compact. Its core is Clarke–Wright savings multiplied by demand compatibility, followed by an explicit capacity-violation penalty and asymmetric depot adjustments.
- **ReEvo** is the smallest solution. It uses inverse distance minus a pairwise demand cost, adds fixed outbound and inbound depot bonuses, and bounds the result with `tanh`.
- **OR-Agent** introduces a qualitatively different geometric pipeline. It reconstructs two-dimensional coordinates from the distance matrix using classical multidimensional scaling, computes angular similarity about the depot, and fuses angular, row-normalized distance, Clarke–Wright savings, and demand-compatibility matrices. The fusion weights are conditioned on demand variance, spatial density, and graph connectivity through a fixed affine-softmax gate. The submitted code computes feature-dependent weights, although the affine parameters themselves are constants rather than trainable parameters.

The closest structural pair is FunSearch–AEL ($S_{\text{AST}} = 0.614$), which is expected because both form large weighted sums of edge-wise distance and demand matrices. Even this pair has operator overlap below 0.5. OR-Agent has the largest operator vocabulary (45 operators), of which 21 are unique among the five programs; these include eigendecomposition, angular functions, median, variance, sorting, stacking, and softmax. Its lowest similarity is with ReEvo (0.120/0.152).

K.3 TSP-Constructive: A Shared Greedy Family with Distinct Realizations

For TSP-CONSTRUCTIVE, the mean pairwise AST similarity is 0.329, with a range of 0.060–0.621, while mean operator overlap is higher at 0.519, with a range of 0.372–0.677 (Table 3). This higher operator overlap has a clear algorithmic explanation: all five programs repeatedly use minimum-edge selection, set copying/removal, loops, and arithmetic accumulation to estimate a continuation tour. Twelve operator types are common to every solution. Thus, the methods converge on the same broad *greedy rollout* family. Nevertheless, their ASTs and decision rules show that they are not merely minor textual variants.

Table 3: Pairwise similarity for TSP-CONSTRUCTIVE. Each entry is $S_{\text{AST}}/S_{\text{op}}$.

	FunSearch	AEL	EOH	ReEvo	OR-Agent
FS	1.000/1.000	0.140/0.455	0.060/0.474	0.173/0.488	0.110/0.372
AEL		1.000/1.000	0.375/0.571	0.621/0.677	0.614/0.567
EOH			1.000/1.000	0.293/0.517	0.417/0.500
ReEvo				1.000/1.000	0.486/0.567
OR-A					1.000/1.000

Table 4: Mechanistic comparison of the TSP-CONSTRUCTIVE solutions.

Generator	Candidate set	Continuation estimate	Exact tail handling	Distinctive decision mechanism
FunSearch	Size-dependent; often nearest 1–2	Recursive, enumerative, or greedy by remaining size	Exact for 2–3 nodes	Piecewise schedule plus a 17-factor large-instance tie-breaker
AEL	All unvisited nodes	Full nearest-neighbor rollout	Exact for at most 3 nodes	Remaining-size-dependent weighting of immediate and future cost
EOH	All unvisited nodes	Full nearest-neighbor rollout	None beyond the one-node base case	Fixed rollout, destination, and relative-distance penalties
ReEvo	All unvisited nodes	Greedy rollout with adaptive score weights	Exact for at most 3 nodes	Remaining-size-dependent lookahead depth and immediate/future balance
OR-Agent	Five nearest nodes	Full nearest-neighbor rollout	None beyond the one-node base case	Subtracts the best simulated 2-opt edge-exchange improvement

The AEL–ReEvo pair has the largest similarity (0.621/0.677). Both score every candidate using immediate distance and a greedy continuation estimate, switch to exact permutation evaluation for a small tail, and adapt the balance according to the number of unvisited nodes. This pair is best described as two close members of the same algorithmic family, rather than independent paradigms.

EOH is the most direct member of that family: for every candidate, it builds a complete nearest-neighbor continuation and combines the rollout with destination and relative-distance penalties. AEL and ReEvo add exact tail evaluation and adaptive weighting. OR-Agent narrows evaluation to the five nearest candidates and augments a greedy continuation with the best counterfactual 2-opt edge exchange. The 2-opt-aware estimate is a distinct local-search operator even though its implementation reuses common loop, subtraction, and comparison primitives.

FunSearch is the structural outlier. Its pairwise AST similarities are only 0.060–0.173, and it contains 641 generated LOC and a branch-point complexity of 145, compared with 63–140 LOC and complexity 7–19 for the other methods. It dispatches among several hand-specialized policies according to the remaining problem size: exact tail enumeration, recursive bounded search, multi-step enumeration with greedy completion, restricted nearest-candidate rollout, and a 17-factor tie-breaker containing connectivity, centrality, bridge, diameter, density, and detour terms.

These results help explain why similar objective values do not imply identical generated programs. Four solutions share a strong nearest-neighbor rollout inductive bias and therefore can converge to similar tours despite different candidate filters, tail solvers, weights, and local-search surrogates. FunSearch is structurally much more elaborate yet performs slightly worse, showing that additional code and branching do not automatically improve the constructive policy. The TSP results therefore exhibit *family-level convergence with implementation-level diversity*, whereas the CVRP results exhibit stronger diversity in both representation and operators.

K.4 Diversity of solutions generated by OR-Agent

We further examine whether OR-Agent converges to the same program across repeated runs. We apply the methodology in Section K.1 to five independently saved TSP-CONSTRUCTIVE solutions. For compact presentation, we denote the runs by R1–R5.

Table 5: Static code statistics for solutions from five OR-Agent runs. The shared import scaffold is excluded.

Run	Objective	LOC	AST nodes	Functions	Branch complexity	Operator vocabulary	Unique ops.
R1	6.240	131	536	4	11	21	1
R2	6.240	146	634	1	22	25	4
R3	6.205	207	1017	5	33	39	6
R4	6.216	206	995	2	28	40	7
R5	6.299	86	312	2	7	17	1

The repeated runs show substantial code-level variation (Table 6). Across the ten pairs, mean AST similarity is 0.445, with a range of 0.270–0.677, and mean operator overlap is 0.449, with a range of 0.386–0.646. The solutions produced by OR-Agent are far from being identical.

Table 6: Pairwise similarity among five OR-Agent runs. Each entry is $S_{\text{AST}}/S_{\text{op}}$.

	R1	R2	R3	R4	R5
R1	1.000/1.000	0.594/0.438	0.408/0.429	0.403/0.386	0.465/0.520
R2		1.000/1.000	0.473/0.391	0.484/0.444	0.395/0.448
R3			1.000/1.000	0.677/0.646	0.277/0.400
R4				1.000/1.000	0.270/0.390
R5					1.000/1.000

All five programs share a broad greedy-lookahead backbone: they tentatively select a next node, estimate the cost of completing the remaining tour, and combine immediate and anticipated costs. Their realizations of this backbone are nevertheless different (Table 7). R1 restricts evaluation to the five nearest nodes and adjusts each full nearest-neighbor completion estimate by the largest saving from a hypothetical single 2-opt exchange. R2 evaluates every node using fixed immediate/future weights and a spatial-dispersion term; when at most 20 nodes remain, it re-evaluates the three leading candidates with a selective two-step lookahead. R3 evaluates all candidates when fewer than five remain and otherwise considers the nearest $\max(5, \lfloor n/2 \rfloor)$ nodes. It estimates continuation cost by adaptively combining a minimum-spanning-tree estimate with a nearest-neighbor rollout and subtracts a distance-variance-based surrogate for future 2-opt potential. R4 evaluates all candidates with mixture weights determined by tour progress and global and local distance statistics, but switches to exact Held–Karp dynamic programming when at most ten nodes remain. R5 is the most compact policy: it evaluates all candidates with a nearest-neighbor completion and adjusts the immediate/future balance only according to the number of unvisited nodes.

Table 7: Mechanistic comparison of solutions produced by five OR-Agent runs.

Run	Candidate set	Continuation estimate	Tail handling	Distinctive decision mechanism
R1	Five nearest nodes	Full nearest-neighbor rollout	One-node base case	Subtracts the best hypothetical single 2-opt saving
R2	All unvisited nodes	Full nearest-neighbor rollout	Selective two-step lookahead for at most 20 nodes	Fixed immediate/future weights plus a spatial-dispersion term
R3	Nearest $\min(n, \max(5, \lfloor n/2 \rfloor))$ nodes	Adaptive MST/nearest-neighbor mixture	Explicit zero- and one-node cases	Distance-variance surrogate for future 2-opt potential
R4	All unvisited nodes	Full nearest-neighbor rollout for $n > 10$	Exact Held–Karp solver for $n \leq 10$	Weights conditioned on progress and global/local distance statistics
R5	All unvisited nodes	Full nearest-neighbor rollout	Zero- and one-node base cases	Immediate/future weights determined by the remaining-node count

The closest pair, R3–R4, has similarity 0.677/0.646. Both are relatively large programs with adaptive lookahead, but their continuation estimators and tail policies differ materially: R3 blends MST and nearest-neighbor estimates, whereas R4 introduces an exact subset dynamic program. Conversely, R4–R5 has the lowest AST similarity (0.270); R5 omits R4’s distance-statistic features and exact tail solver. Similar performance also does not imply an identical implementation: R1 and R2 both attain score around 6.24, yet their AST similarity is 0.594 and their operator overlap is 0.438, reflecting different candidate filters, weighting rules, and refinement mechanisms.

Overall, the repeated runs exhibit *family-level convergence with run-level diversity*. OR-Agent repeatedly discovers the useful inductive bias of greedy tour-completion lookahead, but varies the candidate scope, continuation estimator, adaptive features, exact tail solver, and local-search surrogate. The observed performance range therefore arises from genuinely different executable policies rather than renamings or small constant changes to a single generated program.

L Solution Evolution Case Study

We analyzed solution evolution dynamics from OR-Agent running on BPP_ONLINE problem. The run contains 802 LLM calls, 775 valid responses, 544 function evaluations, 157 saved node-level programs, and 14 strict incumbent updates. The objective improves from 2080.0 to 2014.6, a reduction of 65.4 bins. Relative to the benchmark lower bound of 1996.4, the excess falls from 4.19% to 0.91%, closing 78.23% of the initial gap. OR-Agent ran for 40.86 hours. The final best solution appears after 21.78 hours, 439 LLM responses, and 305 function evaluations. Figure ?? shows that the improvement trajectory is punctuated rather than smooth: the largest single update, $2068.0 \rightarrow 2029.0$ at node 37, accounts for 59.6% of the total gain.

L.1 Program-level evolution

Figure ?? shows the research trees. Table ?? connects the principal performance gains to concrete code changes. The run starts from a 14-line Best-Fit rule that selects the feasible bin with the least residual capacity. Node 14 introduces the recurring backbone used by almost every later incumbent: compute post-placement residuals, reward residuals near multiples of candidate “common sizes,” and retain a lighter Best-Fit term. Nodes 21–35 expand and reweight the multiplier sets, introduce normalization, and condition weights on summary statistics of the open bins. Node 37 is the main conceptual breakthrough: it divides items into small, medium, and large regimes using the item’s percentile among current residual capacities, then assigns regime-specific multiplier sets and mixture weights. This change produces the 39-bin jump.

Round 12 refines that regime-based design rather than replacing it. Node 54 uses dense rational targets for small items and geometric or irrational progressions (including the golden ratio and $\sqrt{2}$) for larger items. Node 63 replaces a uniform closeness score with a piecewise exponential/linear distance kernel. Nodes 70 and 71 return to explicitly enumerated, category-specific target sets and tune their thresholds and weights. Here, a “target set” is a collection of candidate common sizes used to evaluate whether a placement leaves a useful amount of space in the bin. The arrays described as “learned embeddings” in the generated comment text are, in the executable code, fixed hand-coded lookup tables.

Round 23 explores finer representations of target residuals. Node 87 builds a Farey sequence for small-item targets, giving a modest 0.6-bin gain. Node 92 then makes the target generator context-dependent. Its docstring describes kernel-density estimation, but the executable mechanism is simpler: it forms candidate common sizes from quantiles of the currently feasible post-placement residuals, ratios of the current item, fractions of residual capacities, and a limited set of item–residual cross-ratios. It preserves percentile regimes and the inverse-distance multiple-fit score. This substitution of a dynamic target set for node 88’s static set yields the final 4.2-bin improvement.

L.2 Recurring structures and structural diversity

In this subsection, we make use of code statistics like branch complexity and structural continuity to analyze code. Branch complexity starts at one and adds one for each branch, loop, comprehension, exception handler, assertion, or Boolean decision. Structural continuity is measured with multiset Jaccard similarity over normalized depth-two AST subtrees, with constants normalized by type. Thus, the measure captures local control and expression structure while discounting literal and formatting changes.

The trajectory has a stable algorithmic spine rather than repeatedly starting from scratch. All 14 incumbents contain a Best-Fit preference; 13 of 14 also use post-placement residuals and common-size/multiple-fit scoring. Percentile regimes occur in the last seven incumbents. Geometric targets, an exponential kernel, a Farey construction, and dynamic remainder-derived targets each appear as more specialized mutations of this spine.

This recurrence coexists with meaningful structural change. Across 50 parent–child edges in the five principal trees, mean normalized AST similarity is 0.768 (median 0.764, range 0.285–1.000). Consequently, most children are recognizable local revisions, but some branches perform substantial restructuring. For example, the parent–child similarities of the successful node-54 and node-92 transitions are 0.656 and 0.509, respectively, whereas the refinement from node 26 to node 27 has similarity 0.931.

Code complexity is likewise non-monotone. Generated code grows from 14 lines and branch complexity 1 at the initial Best-Fit rule to a maximum of 268 lines at node 71, then contracts to 235 lines at node 87 and 169 lines at node 92. Branch complexity nevertheless rises to 20 in the final incumbent because the shorter program concentrates its logic in percentile cases and bounded candidate construction. OR-Agent therefore alternates expansion, specialization, and pruning rather than exhibiting uniform code bloat.

L.3 Reflections steering revisions

As shown by Table ??, at node 21, reflection diagnoses that a sliding-window history is ineffective because the evaluation setting does not provide useful persistent state. It replaces history with stateless item-derived multiples, reduces the Best-Fit weight, and improves $2087.0 \rightarrow 2080.2 \rightarrow 2077.8$. Removing one multiplier then worsens the score to 2078.2, so reflection requests a rollback. Nodes 54, 63, and 71 show the same pattern: profitable changes are retained, while a later regression is reverted. At node 92, the dynamic generator immediately produces 2014.6; a subsequent revision returns 2080.2 and later trials time out, so the first version remains the incumbent.

L.4 Summary

The solution evolution trajectory has a stable algorithmic spine rather than repeatedly starting from scratch. Reflection consistently preserves improvements and rolls back regressions, while periodic rerooting and exploratory proposals prevent the search from collapsing to a single lineage. Code size and branch complexity evolve non-monotonically, reflecting alternating expansion, specialization, and pruning rather than uniform code growth.

M Baseline-Surpassing Solution of Cooperative Driving Problem

In this section we explain the idea behind the baseline-Surpassing cooperative driving algorithm found by OR-Agent. It’s a centralized cooperative driving controller that jointly determines the longitudinal acceleration and lane-changing actions of all vehicles within the controlled road segment. The principal objective is to proactively create safe merging opportunities, allocate these opportunities among competing vehicles, and verify the consistency of the resulting traffic configuration before executing any commands. Unlike approaches that separately handle car following, platoon formation, lane swapping, and individual lane-change maneuvers, the proposed controller organizes these decisions into a unified pipeline. At each simulation step, it first computes collision-aware longitudinal speeds, identifies and prioritizes vehicles requiring lane changes, evaluates merge feasibility, reserves feasible merge regions, coordinates gap creation for infeasible requests, and finally performs a network-wide safety validation.

Collision-aware car following Longitudinal motion is governed by a collision-aware car-following model. Vehicles in each lane are ordered from downstream to upstream, allowing the closest leader of every following vehicle to be identified. The next-step speed of vehicle i is constrained by the road speed limit, its acceleration capability, and a braking-safe speed:

$$v_i^{t+1} = \min \{v_{\max}, v_i^t + a_i^{\max} \Delta t, v_i^{\text{safe}}\}, \quad (3)$$

where v_i^t denotes the current speed, $a_i^{\max}$ is the maximum acceleration, $v_{\max}$ is the road speed limit, and Δt is the simulation time step.

The braking-safe speed accounts for the current gap g_i , the leader speed v_ℓ , the vehicle-specific maximum deceleration b_i , a reaction-time allowance τ , and the minimum desired following gap $g_{\min}$:

$$v_i^{\text{safe}} = -b_i \tau + \sqrt{(b_i \tau)^2 + v_\ell^2 + 2b_i \max(0, g_i - g_{\min})}. \quad (4)$$

This formulation restricts the follower to a speed from which it can respond safely if its leader decelerates. Vehicle-specific acceleration, deceleration, and length parameters are used to represent the different dynamic characteristics of passenger cars, trucks, buses, and emergency vehicles.

Route-aware lane requirements Lane-change demand is determined from route information. For each vehicle, the controller identifies the route-compatible target lane from the set of potential downstream lanes. Vehicles already occupying their required lane are excluded from the lane-change procedure, whereas vehicles in an incompatible lane are treated as merge candidates.

The urgency of vehicle i is defined as a function of the remaining distance d_i to the end of its current lane:

$$u_i = \max \left[0, \min \left(1, 1 - \frac{d_i}{d_{\text{urg}}} \right) \right], \quad (5)$$

where d_{urg} denotes the distance over which lane-change urgency develops. Candidates are ordered according to decreasing urgency and, when urgency values are equal, decreasing longitudinal position. This prioritization ensures that vehicles approaching a mandatory exit or lane termination are processed before vehicles with greater remaining maneuvering distance.

Entry-zone protection An entry-zone protection mechanism is introduced because recently arriving vehicles may not yet be observable when lane-change decisions are computed. A vehicle is therefore prevented from initiating a lane change during a short interval after entering the controlled segment and while it remains within a specified distance of the entry boundary. In the evaluated configuration, these thresholds were 2 s and 30 m, respectively. This restriction prevents a vehicle from merging into a lane immediately before another vehicle enters that lane from an upstream edge.

Front-and-rear merge safety For each eligible lane-change candidate, the controller identifies the closest leader and follower in the adjacent target lane. Let g_i^{front} and g_i^{rear} denote the front and rear clearances associated with candidate vehicle i . The corresponding time-to-collision values are calculated as

$$\text{TTC}_i^{\text{front}} = \begin{cases} \frac{g_i^{\text{front}}}{v_i - v_\ell}, & v_i > v_\ell, \\ \infty, & v_i \leq v_\ell, \end{cases} \quad (6)$$

and

$$\text{TTC}_i^{\text{rear}} = \begin{cases} \frac{g_i^{\text{rear}}}{v_f - v_i}, & v_f > v_i, \\ \infty, & v_f \leq v_i, \end{cases} \quad (7)$$

where v_ℓ and v_f are the speeds of the target-lane leader and follower, respectively.

The required merge clearance is moderately adjusted according to urgency:

$$g_i^{\text{merge}} = g_{\text{base}} - \alpha_u u_i, \quad (8)$$

where g_{base} is the nominal merge clearance and α_u controls the degree of urgency-based relaxation. In the evaluated configuration, $g_{\text{base}} = 6$ m. A lane change is feasible only if

$$\begin{aligned} g_i^{\text{front}} &\geq g_i^{\text{merge}}, \\ g_i^{\text{rear}} &\geq g_i^{\text{merge}}, \\ \text{TTC}_i^{\text{front}} &\geq \text{TTC}_{\text{min}}, \\ \text{TTC}_i^{\text{rear}} &\geq \text{TTC}_{\text{min}}. \end{aligned} \quad (9)$$

A lane-change cooldown and the entry-zone restrictions must also be satisfied. Although urgency can moderately reduce the required physical clearance, it does not override the TTC constraints.

Merge reservations Because multiple vehicles can independently identify the same target-lane opening, feasible maneuvers are supplemented by a reservation mechanism. Once a lane-change request is approved, the controller reserves a longitudinal region around the intended merge position. A subsequent vehicle j cannot reserve the same target lane if

$$|x_i - x_j| < d_{\text{res}}, \quad (10)$$

where x_i and x_j are the intended merge positions and d_{res} is the reservation distance. The reservation mechanism converts simultaneous lane-change selection into an ordered allocation process and prevents vehicles approaching from different source lanes from being assigned to the same physical gap.

Cooperative gap creation When an immediately safe gap is unavailable, the controller attempts to create one cooperatively rather than forcing the requesting vehicle to stop. If the front clearance is insufficient, the merging vehicle reduces its speed while the target-lane leader is permitted to accelerate, thereby enlarging the gap ahead. If the rear clearance is insufficient, the target-lane follower yields by decelerating, while the merging vehicle may accelerate toward the developing slot.

The cooperative maneuver therefore coordinates three vehicles:

$$\{\text{target-lane leader, merging vehicle, target-lane follower}\}. \quad (11)$$

This mechanism is particularly important under high traffic demand, where waiting for naturally occurring large gaps can substantially reduce throughput or produce gridlock.

Post-coordination synchronization Cooperative adjustments are interdependent because a vehicle may participate in several local interactions during the same simulation step. A speed modification introduced by a later request can invalidate a merge that was approved earlier. The controller therefore performs a post-coordination synchronization stage after processing all lane-change requests.

For every approved maneuver, the merging vehicle is constrained so that it does not approach its new leader excessively. Similarly, its new follower is constrained to remain slightly slower during the lateral transition. These relationships can be expressed as

$$v_i^{t+1} \leq v_\ell^{t+1} - \Delta v_{\text{sync}}, \quad (12)$$

and

$$v_f^{t+1} \leq v_i^{t+1} - \Delta v_{\text{sync}}, \quad (13)$$

where $\Delta v_{\text{sync}} > 0$ is a synchronization margin. This second evaluation ensures that the final speeds, rather than the preliminary speeds used during initial approval, remain compatible with the reserved merge.

Projected dual-lane safety validation During a lateral transition, a lane-changing vehicle can interact with traffic in both its source lane and its target lane. The controller therefore constructs a projected traffic configuration in which each lane-changing vehicle temporarily occupies both lanes. Vehicles in every projected lane are ordered from front to back, and the one-step gap between each leader-follower pair is evaluated.

For a leader ℓ and follower f , the projected gap is approximated as

$$\hat{g}_f^{t+1} = g_f^t + (v_\ell^{t+1} - v_f^{t+1}) \Delta t. \quad (14)$$

To maintain the minimum following clearance, the follower speed is constrained by

$$v_f^{t+1} \leq v_\ell^{t+1} + \frac{g_f^t - g_{\min}}{\Delta t}. \quad (15)$$

When this constraint is violated, the follower speed is reduced subject to its physical deceleration limit. The procedure is repeated for several iterations so that braking adjustments propagate upstream through the traffic queue. This validation prevents a local cooperative maneuver from creating a rear-end conflict elsewhere in the system.

Overall decision pipeline The complete control process comprises seven sequential stages:

1. computation of collision-aware longitudinal speeds;
2. prioritization of route-dependent merge requests;
3. validation of the available gaps and front and rear TTC conditions;
4. reservation of feasible target-lane merge regions;
5. cooperative creation of gaps for currently infeasible requests;

6. synchronization of vehicle speeds after all cooperative adjustments; and
7. projected network-wide safety validation across both source and target lanes.

By combining local safety constraints with system-level coordination, it enables mandatory lane changes while avoiding the excessively conservative spacing requirements that can reduce traffic flow under high demand.

N Sensitivity Analysis of OR-Agent Solutions

In this appendix section, we conduct sensitivity analysis of OR-Agent solutions for cooperative driving and CVRP-LEHD problems.

N.1 Sensitivity Analysis of “Spaghetti Code” and Baseline Surpassing Solutions for Driving Problem

We compare the sensitivity for two solutions found by OR-Agent for cooperative driving problem: “spaghetti code” solution and baseline-surpassing solution. A direct comparison based on the average sensitivity over all declared parameters can be misleading when the controllers have different parameter counts. In particular, arbitrarily adding unused parameters would reduce the average sensitivity even though the controller behavior remains unchanged. We therefore employed an activity-aware analysis that separates influential, locally dormant, and structurally unused parameters. The analysis compared the tuned spaghetti controller with the final cooperative controller while keeping both implementations of `driving_actions` functions fixed.

Experimental design The spaghetti-code and baseline-surpassing controllers contained 23 and 9 declared scalar hyperparameters, respectively. For each parameter p_j , one-at-a-time perturbations of $\pm 10\%$ were applied around its baseline value p_j^0 :

$$p_j^- = 0.9p_j^0, \quad p_j^+ = 1.1p_j^0. \quad (16)$$

All remaining parameters were held at their baseline values. Both the low-demand and high-demand traffic cases were simulated for every perturbation. Including the baseline evaluations, the one-at-a-time experiments, and the joint experiments described below, the analysis comprised 114 complete controller evaluations.

Let S_0 denote the baseline aggregate score and let S_j^- and S_j^+ denote the scores obtained from the negative and positive perturbations of parameter p_j . The local sensitivity magnitude of p_j was defined as

$$s_j = \frac{|S_j^- - S_0| + |S_j^+ - S_0|}{2}. \quad (17)$$

This definition accommodates asymmetric and discontinuous responses, which were common in the spaghetti controller.

A parameter was classified as *score-active* if either perturbation changed the aggregate score beyond a numerical tolerance of 10^{-9} . A parameter was classified as *metric-only active* if it changed at least one reported traffic metric without changing the aggregate score. Parameters referenced by the controller but producing no observed response were classified as *locally dormant*. Finally, parameters with no executable read after their assignment were classified as *structurally unused*. Executable references were identified from the abstract syntax tree rather than by textual matching.

Sensitivity measures robust to spurious parameters The primary local sensitivity measure was the mean effect conditional on the score-active parameter set $\mathcal{A}$:

$$\bar{s}_{\text{active}} = \frac{1}{|\mathcal{A}|} \sum_{j \in \mathcal{A}} s_j. \quad (18)$$

Unlike the mean over all declared parameters, this quantity is unchanged if arbitrary zero-effect parameters are appended to the controller.

We additionally calculated the total local effect

$$S_{L1} = \sum_{j=1}^P s_j \quad (19)$$

and the effective sensitivity dimension

$$d_{\text{eff}} = \frac{\left(\sum_{j=1}^P s_j\right)^2}{\sum_{j=1}^P s_j^2}. \quad (20)$$

Both measures are invariant to the addition of exact zero-effect parameters. The effective dimension indicates whether the total sensitivity is concentrated in a few dominant parameters or distributed across many parameters.

Joint perturbation analysis One-at-a-time experiments quantify local main effects but may not reveal parameter interactions. We therefore conducted 24 deterministic random joint trials for each controller. In trial r , every structurally referenced parameter was independently perturbed according to

$$p_j^{(r)} = p_j^0 \left(1 + \epsilon_j^{(r)}\right), \quad \epsilon_j^{(r)} \sim \mathcal{U}(-0.1, 0.1). \quad (21)$$

Structurally unused parameters were excluded from the joint perturbations, whereas locally dormant but referenced parameters were retained because they could participate in higher-order interactions. Joint sensitivity was summarized using the standard deviation and range of the resulting scores. A normalized joint standard deviation was also calculated by dividing the score standard deviation by $\max(|S_0|, 1)$.

Parameter activity Of the 23 declared spaghetti-code controller parameters, 22 were referenced by executable code and 18 were score-active under the local perturbations. Four parameters were locally dormant. The parameter `PLATOON_ACCEL` was structurally unused. For the final controller, 7 of the 9 parameters were score-active, while `FOLLOW_GAP` and `REACTION_TIME` were locally dormant. No metric-only active parameters were observed.

Local sensitivity results The activity-aware results are summarized in Table 8. The active-parameter mean sensitivity of the spaghetti-code controller was 43.855 score points, compared with 15.310 for the baseline-surpassing controller. Thus, conditional on a parameter affecting the score, the average response of the spaghetti-code controller was 2.864 times larger.

The spaghetti-code controller also exhibited a total L1 effect of 789.392, which was 7.366 times the corresponding value of 107.170 for the baseline-surpassing controller. Its effective sensitivity dimension was 12.764, compared with 3.168 for the final controller. Consequently, the spaghetti-code controller was not only more sensitive on average, but its sensitivity was also distributed across a substantially larger number of influential parameter directions.

Table 8: Activity-aware local and joint sensitivity measures.

Controller	$\bar{s}_{\text{active}}$	S_{L1}	d_{eff}	σ_{joint}
Spaghetti-code	43.855	789.392	12.764	44.183
Baseline-surpassing	15.310	107.170	3.168	28.050

The most sensitive spaghetti-code controller parameter was `PLATOON_DECEL`, with a mean absolute score change of 114.912. It was followed by `COOPERATION_RANGE` (89.166), `SWAP_COORDINATION_DECEL` (70.741), `COOPERATION_WINDOW` (65.430), and `SPEED_MATCH_FACTOR` (55.535). The dominant baseline-surpassing controller parameters were `COOP_BRAKE` (50.528), `ENTRY_MERGE_DISTANCE` (25.972), and `ENTRY_DELAY` (17.588).

Many responses were strongly asymmetric. For example, decreasing `PLATOON_DECEL` resulted in a score of -95.510 , while increasing it resulted in a score of 15.945 . These results indicate thresholded and discontinuous behavior that would not be adequately represented by a single central derivative.

Joint sensitivity and robustness The standard deviation of the spaghetti-code controller scores under joint perturbations was 44.183, compared with 28.050 for the baseline-surpassing controller, corresponding to an absolute variability ratio of 1.575. After normalization by the baseline-score magnitude, the standard deviations were 0.588 and 0.292, respectively, giving a normalized ratio of 2.013.

The spaghetti-code joint-perturbation scores ranged from -98.575 to 33.758 , and none of the 24 trials satisfied the feasibility criterion. The mean high-demand collision count was 5.417, and the mean high-demand speed was 2.485 m s^{-1} . In contrast, the baseline-surpassing controller scores ranged from 16.856 to 95.969, and 29.2% of the joint trials remained feasible. Its mean high-demand collision count and speed were 0.542 and 6.764 m s^{-1} , respectively. These results show that the tuned spaghetti-code controller occupies a considerably narrower region of satisfactory parameter space.

Summary Overall, the active-parameter mean, total L_1 effect, effective sensitivity dimension, and both absolute and normalized joint variability consistently indicate that the spaghetti-code controller is substantially more hyperparameter-sensitive than the baseline-surpassing controller. Importantly, this conclusion is not caused by the spaghetti-code controller having more declared parameters, because inactive and structurally unused parameters were excluded from the primary average and could not dilute the L_1 or effective-dimension measures.

As to the limitations of analysis above, parameters classified as locally dormant may become active under larger perturbations, and the finite joint trials do not constitute an exhaustive worst-case analysis. The results are also specific to the two supplied traffic-demand scenarios, the simulation seed, the current tuned baselines, and the employed evaluation function.

N.2 Sensitivity Analysis for CVRP–LEHD Problem

Experimental design We evaluated the local hyperparameter sensitivity of the final CVRP–LEHD solution using the original evaluation procedure without modifying the solution logic. We retained greedy decoding, checkpoint 40, and the evaluator seed of 12, with RRC disabled. The reported objective, denoted by S , is the unweighted mean route length over the two problem sizes and is therefore minimized. The unperturbed solution obtained route lengths of 20.567616 and 38.107460 for $n = 200$ and $n = 500$, respectively, giving $S_0 = 29.338$.

The raw fusion layer contains 12 feature-to-logit coefficients and four biases, in addition to the angular bandwidth. However, the softmax output is invariant to a common shift of the four logits. This creates three redundant coefficient directions—one for each input feature—and one redundant bias direction. To prevent these non-identifiable directions from artificially diluting the sensitivity estimate, we fixed the demand component as the reference. The resulting analysis therefore contains 13 identifiable dimensions: one angular bandwidth, nine relative feature-to-logit coefficients, and three relative biases. Numerical safeguards such as 10^{-8} and fixed structural constants were not treated as hyperparameters.

We first performed one-at-a-time (OAT) perturbations, changing each identifiable parameter by -10% and $+10\%$ while holding the others fixed. For parameter j , let S_j^- and S_j^+ denote the two perturbed scores. Its local sensitivity was defined as

$$I_j = \frac{1}{2} (|S_j^- - S_0| + |S_j^+ - S_0|). \quad (22)$$

A parameter was classified as score-active when $I_j > 10^{-7}$; otherwise, it was classified as locally dormant. This terminology is deliberately local: because greedy decoding uses an argmax decision, the response surface is piecewise constant and discontinuous, and a parameter that is inactive within the tested neighborhood can become active under a larger perturbation or on a different instance set.

To avoid sensitivity estimates that can be reduced merely by appending inactive or spurious parameters, we report the mean response conditional on the score-active set $\mathcal{A}$, the total L_1 effect $\sum_j I_j$, and the effective sensitivity dimension

$$d_{\text{eff}} = \frac{\left(\sum_j I_j\right)^2}{\sum_j I_j^2}. \quad (23)$$

These quantities are unchanged if exact zero-effect parameters are appended. Finally, we conducted 16 joint trials in which every identifiable parameter was independently multiplied by a factor sampled uniformly from $[0.9, 1.1]$, using random seed 20261005. The OAT and joint experiments required 43 evaluator runs in total.

One-at-a-time results Table 9 reports the full OAT ranking. To keep the table compact, v_d , ρ , and κ denote demand variance, spatial density, and connectivity, respectively; “ang,” “dist,” and “sav” denote the angular, distance, and savings components. The quantities $I_j^{(200)}$ and $I_j^{(500)}$ are computed as in Eq. (22), but separately for each problem size.

Table 9: One-at-a-time sensitivity of the CVRP-LEHD solution to $\pm 10\%$ hyperparameter perturbations. Lower scores are better. A dagger marks a parameter that was locally dormant within the tested neighborhood.

Rank	Parameter	I_j	S_j^-	S_j^+	$I_j^{(200)}$	$I_j^{(500)}$
1	$w_{\text{ang}, \rho}$	0.018655	29.337675	29.374710	0.015589	0.022202
2	σ_{ang}	0.013424	29.364008	29.337915	0.022589	0.004259
3	$w_{\text{dist}, \rho}$	0.010020	29.354818	29.340298	0.000378	0.019663
4	w_{ang, v_d}	0.002075	29.337915	29.341311	0.000378	0.003773
5	w_{dist, v_d}	0.002075	29.337538	29.341688	0.000378	0.003773
6	w_{sav, v_d}	0.002075	29.337538	29.341688	0.000378	0.003773
7	$w_{\text{sav}, \rho}$	0.002075	29.337538	29.341688	0.000378	0.003773
8	$w_{\text{ang}, \kappa}$	0.001886	29.337538	29.341311	0.000000	0.003773
9	$w_{\text{sav}, \kappa}$	0.001886	29.337538	29.341311	0.000000	0.003773
10	$w_{\text{dist}, \kappa}$	0.000189	29.337538	29.337915	0.000378	0.000000
11	b_{dist}	0.000189	29.337538	29.337915	0.000378	0.000000
12	$b_{\text{ang}}^\dagger$	0.000000	29.337538	29.337538	0.000000	0.000000
13	$b_{\text{sav}}^\dagger$	0.000000	29.337538	29.337538	0.000000	0.000000

Of the 13 identifiable dimensions, 11 were score-active and two were locally dormant. The mean and median values of I_j over the active set were 0.004959 and 0.002075, respectively. The total L_1 effect was 0.054550, and the effective sensitivity dimension was 4.557, indicating that the local response was concentrated in a relatively small subset of the identifiable parameter space. The largest response was associated with the angular spatial-density coefficient, followed by the angular bandwidth and the distance spatial-density coefficient. Even the largest mean OAT response was only 0.0636% of the baseline score.

The problem-size-specific effects reveal some heterogeneity that would be hidden by considering only the aggregate objective. For example, the angular bandwidth had a larger effect at $n = 200$, whereas the distance spatial-density coefficient primarily affected $n = 500$. Reporting both sizes is therefore important because aggregation can otherwise obscure size-dependent effects or partial cancellation.

Joint-perturbation results Table 10 summarizes the joint trials. The mean perturbed score was 29.349978, and the sample standard deviation was 0.009026, corresponding to 0.0308% of the baseline. The scores ranged from 29.337297 to 29.362148. The best sampled change was an improvement of 0.000240, whereas the worst sampled degradation was 0.024611, or 0.0839% of the baseline. All 16 trials remained within 0.1% of the baseline score.

Summary Taken together, the OAT and joint experiments indicate that the final CVRP-LEHD solution is locally robust within the tested $\pm 10\%$ neighborhood. This conclusion is based on identifiable, activity-aware quantities and therefore cannot be made artificially stronger by adding unused or redundant hyperparameters. This result should not be interpreted as a global robustness guarantee; locally dormant parameters can become active for larger changes or different instances.

Table 10: Aggregate OAT and joint-perturbation sensitivity statistics.

Statistic	Value
Identifiable dimensions	13
Score-active dimensions	11
Locally dormant dimensions	2
Active-set mean I_j	0.004959
Active-set median I_j	0.002075
Total L_1 effect	0.054550
Effective sensitivity dimension	4.557
Maximum OAT effect / baseline	0.0636%
Joint-score sample standard deviation	0.009026
Joint standard deviation / baseline	0.0308%
Joint mean absolute score change	0.012500
Worst joint degradation / baseline	0.0839%
Joint-trial improvement rate	12.5%
Joint trials within 0.1% of baseline	100.0%